



Cairo University

Faculty of Computing and Artificial Intelligence

Department of Computer Science



Game Forge

**AI-Powered Natural Language to Playable Game
Generation Platform**

Graduation Project — Final Documentation

Academic Year: 2025 / 2026

Supervisors:

Dr. Samar Hesham

TA. Nada Nasser

Team Members:

1. Ahmed Saber Ahmed

ID: 20220021

2. Omar Mohamed AbdElmelek

ID: 20220232

3. Amr Mohamed Sheraey

ID: 20220450

4. Omar Mahfouz Mohamed

ID: 20220229

Abstract

English

Game development is a multidisciplinary and resource-intensive process that requires the integration of programming, game engine expertise, visual and audio asset creation, game design, and continuous iteration. This complexity poses a significant challenge for solo developers, students, and small teams, often preventing promising game ideas from progressing beyond early prototypes. Although recent advances in large language models and generative technologies have enabled partial automation of game design, code generation, and asset creation, existing solutions typically address these components in isolation and lack a unified, end-to-end workflow.

Game Forge is an end-to-end AI-powered platform that transforms a natural-language description into a fully playable browser game in a single automated pipeline. Users describe their desired game in plain text; the system classifies the prompt into one of three supported archetypes (platformer, top-down, or endless-runner), generates a structured game plan using a large language model, produces sprite and audio assets through the Stability AI and Stable Audio APIs, writes syntactically valid GDScript code for the Godot 4 engine, compiles an HTML5 export via the Godot headless CLI, and streams the playable result back through the browser. The platform is built as ten cooperating Docker microservices including a React frontend, a Node.js/Express REST backend, a stateless orchestration server, four specialized worker services, an MCP-based asset generation server, MongoDB for persistence, and MinIO for object storage. Key technical contributions include a template-guided code generation pipeline with Godot syntax validation, an LLM-driven MCP agent for asset orchestration, and a real-time build progress system. The platform successfully generates playable games across all three supported genres from a single user prompt.

The proposed system aims to lower the barrier to entry for game development by providing an integrated, automated pipeline that converts high-level ideas into functional game prototypes.

Arabic

تُعدّ تطوير الألعاب عملية متعددة التخصصات وتستلزم موارد ضخمة، إذ تتطلب التكامل بين البرمجة، والإلمام بمحركات الألعاب، وإنشاء الأصول المرئية والصوتية، وتصميم اللعبة، والتكرار المستمر. ويُشكّل هذا التعقيد تحدياً جسيماً أمام المطورين المنفردين والطلاب والفرق الصغيرة، إذ كثيراً ما يحول دون تقدم أفكار الألعاب الواعدة إلى ما هو أبعد من النماذج الأولية المبكرة. وعلى الرغم من أن التقدم الأخير في نماذج اللغة الكبيرة والتقنيات التوليدية قد أتاح أتمتةً جزئيةً لتصميم الألعاب وتوليد الكود وإنشاء الأصول، إلا أن الحلول القائمة تعالج هذه المكونات في الغالب بصورة منفصلة، وتفتقر إلى سير عمل موحد ومتكامل من البداية إلى النهاية.

جيم فورج هي منصة متكاملة مدعومة بالذكاء الاصطناعي تُحوّل وصفاً بالغة الطبيعية إلى لعبة متصفح قابلة للعب بالكامل عبر خط أنابيب آلي متسلسل. يصف المستخدمون لعبتهم المطلوبة بلغة بسيطة، فيقوم النظام بتصنيف النص ضمن أحد الأنماط الثلاثة المدعومة (منصة جانبية، أو منظور علوي، أو ماش لا نهائي)، ثم يولّد خطة لعبة منظمة Stability AI وStable Audio باستخدام نموذج لغوي كبير، وينتج أصول الرسوم المتحركة والصوت عبر واجهتيّ عبر واجهة HTML5 ، ويصدّر اللعبة بصيغة Godot 4 صحيحة نحويّاً لمحرك GDScript ، ثم يكتب شفرة Audio بدون واجهة رسومية، ويبث النتيجة القابلة للعب مباشرةً في المتصفح. تتكون المنصة من عشره خدمات Godot ، وخادم Node.js/Express — REST الأمامية، وخادم React تشمل واجهة Docker مصغرة في حاويات MCP ، لتنسيق عديم الحالة، وأربع خدمات عمال ذكاء اصطناعي متخصصة، وخادم توليد أصول مبني على بروتوكول تشمل المساهمات التقنية الرئيسية خط أنابيب MinIO للاستمرارية، وتخزين كائنات MongoDB وقاعدة بيانات مدفوع بنموذج لغوي لتنسيق الأصول، MCP ، ووكيل Godot توليد كود موجّه بالقوالب مع التحقق من صحة بنية ونظام تتبع تقدم البناء في الوقت الفعلي. وتنتج المنصة في توليد ألعاب قابلة للعب عبر الأنماط الثلاثة المدعومة كلها انطلاقاً من نص واحد مُدخّل من المستخدم.

يهدف النظام المقترح إلى تخفيض حاجز الدخول إلى مجال تطوير الألعاب من خلال توفير خط أنابيب آلي متكامل يُحوّل الأفكار على مستوى عالٍ إلى نماذج أولية وظيفية للألعاب.

Table of Contents

Chapter 1: Introduction.....	1
1.1 Background and Motivation	1
1.2 Problem Definition	1
1.3 Project Objectives and Proposed Solution.....	2
1.4 Project Scope and Limitations	3
1.5 Development Methodology	3
1.6 Tools and Technologies Used	4
1.7 Project Timeline / Gantt Chart.....	5
1.8 Report Organization.....	6
Chapter 2: Related Work.....	7
2.1 Prompt-to-Game Platforms	7
2.1.1 Rosebud AI	7
2.1.2 Playabl.ai.....	7
2.1.3 Astrocade	7
2.1.4 StoneLeap.....	8
2.2 AI Asset and Design Tools	8
2.2.1 Ludo.ai	8
2.2.2 GDevelop AI.....	9
2.3 General-Purpose Code Generation	9
2.3.1 LLM Code Assistants (GitHub Copilot and Similar).....	9
2.3.2 Procedural Content Generation Systems	9
2.4 Comparative Analysis	10
Chapter 3: System Analysis	11
3.1 Stakeholder Analysis	11
3.2 Functional Requirements.....	11
3.2.1 User Authentication and Account Management	11
3.2.2 User Profile and Project Management	11
3.2.3 Prompt Submission and Validation	12
3.2.4 Game Planning	12
3.2.5 Asset Generation and Management.....	12
3.2.6 Code and Scene Generation.....	12

3.2.7 Build and Execution	12
3.2.8 Game Access and Distribution	13
3.2.9 Status Reporting and Error Handling	13
3.3 Non-Functional Requirements	13
3.3.1 Security	13
3.3.2 Performance.....	14
3.3.3 Scalability	14
3.3.4 Reliability	15
3.3.5 Availability	15
3.3.6 Maintainability	15
3.3.7 Usability.....	16
3.3.8 Portability	16
3.4 Use Case Diagrams.....	16
3.5 Feasibility Study.....	19
3.5.1 Technical Feasibility	19
3.5.2 Economic Feasibility	19
3.5.3 Operational Feasibility.....	19
Chapter 4: System Design	20
4.1 System Architecture.....	20
4.2 Class Diagrams	22
4.3 Sequence Diagrams	26
4.4 Entity-Relationship Diagram	34
4.5 UI Wireframes / Mockups	35
4.6 Security Design Considerations.....	40
4.6.1 Authentication and Authorization.....	40
4.6.2 Inter-Service Authentication	40
4.6.3 Input Sanitization.....	40
4.6.4 Password Security	40
4.6.5 Rate Limiting	40
4.6.6 Cross-Origin Resource Sharing	41
4.6.7 Environment Variable Security	41
Chapter 5: Implementation	41

5.1 Implementation Environment and Setup	41
5.2 Key Implementation Details	42
5.2.1 Game Plan Generation (Planner Service)	42
5.2.2 Asset Generation via MCP (Asset Service)	43
5.2.3 Template-Guided Code Generation (Code Service)	44
5.2.4 Builder Service and Godot Export	44
5.2.5 Real-Time Progress Tracking	45
5.3 Sample Screenshots / Output Samples	46
Chapter 6: Testing and Evaluation	49
6.1 Testing Strategy	49
6.1.1 Testing Scope by Service	50
6.1.2 Toolchain Decisions	50
6.1.3 Mocking and Isolation Strategy	51
6.2 Testing Cases and Results	52
6.2.1 Code Coverage	53
6.3 Performance Evaluation / Benchmarking	54
6.3.1 Planner Service — Latency and Plan Quality	54
6.3.1.1 Latency Results	55
6.3.1.2 Quality Evaluation	56
6.3.2 Code Assembly Service — Throughput vs. Plan Complexity	56
6.3.2.1 Latency Results	56
6.3.3 Asset Generation Service — Stability AI Throughput	59
6.3.4 End-to-End Pipeline — Stage Breakdown	60
6.3.5 Real-World Production Telemetry	61
6.3.5.1 Deployment Overview	62
6.3.5.2 Generation Pipeline Performance	62
6.3.5.3 Build & Deliverable Metrics	65
6.3.5.4 Content Generation Metrics	66
6.3.5.5 Reconciliation with Estimated Benchmarks and Key Findings	67
6.4 Limitations and Known Issues	68
6.4.1 Game Scope and Design Generation	68
6.4.1.1 Only Three Game Archetypes	68

6.4.1.2 Fixed Behavior and Weapon Catalog	68
6.4.1.3 Generic Fallback Plan.....	69
6.4.2 Visual and Audio Asset Generation.....	69
6.4.2.1 Asset Generation Is the Pipeline Bottleneck.....	69
6.4.2.2 Dependence on External Image Credits	69
6.4.2.3 No Frame-by-Frame Sprite Animation	70
6.4.2.4 Art Style Is Not Always Exact	70
6.4.2.5 No Asset Caching	70
6.4.3 Code Assembly	70
6.4.3.1 Template-Bounded Generation	70
6.4.3.2 The Repair Loop Degrades Rather Than Repairs	70
6.4.3.3 Bounded Level Variety.....	71
6.4.4 Build and Export.....	71
6.4.4.1 Environment-Sensitive Godot Export.....	71
6.4.5 Orchestration, Scale, and Reliability	71
6.4.5.1 No Partial Recovery or Cancellation	71
6.4.5.2 Polling-Based Progress and Best-Effort Status	71
6.4.6 Authentication and Security.....	72
6.4.6.1 Shared Inter-Service Secret	72
6.4.6.2 Publicly Addressable Game Files	72
Chapter 7: Conclusion and Future Work	72
7.1 Summary of Achievements.....	72
7.2 Lessons Learned	75
7.3 Future Work.....	77
References.....	I

List of Figures

Figure 3.1 — Use Case Diagram	18
Figure 4.1 — System Architecture / Component Diagram.....	21
Figure 4.2 — Class Diagram: Core MongoDB Models	22
Figure 4.3 — Class Diagram: Code Pipeline Classes	23
Figure 4.4 — Class Diagram: Asset MCP Server Classes	24
Figure 4.5 — Class Diagram: Backend Service Classes.....	25
Figure 4.6 — Sequence Diagram: User Registration and Login.....	26
Figure 4.7 — Sequence Diagram: Create Project and Chat Refinement	27
Figure 4.8 — Sequence Diagram: Full Game Generation Pipeline	28
Figure 4.9 — Sequence Diagram: Asset Generation via MCP	29
Figure 4.10 — Sequence Diagram: Code Generation with Validation Loop	30
Figure 4.11 — Sequence Diagram: Builder Service and Godot Export	31
Figure 4.12 — Sequence Diagram: Real-Time Progress Tracking	32
Figure 4.15 — Wireframe: Authentication (Login / Signup)	35
Figure 4.16 — Wireframe: Workspace — New Game Prompt State	36
Figure 4.17 — Wireframe: Workspace — Projects and Chat Panel	37
Figure 4.18 — Wireframe: Build Panel — Success State.....	38
Figure 4.19 — Wireframe: Play Game View.....	39
Figure 4.20 — Wireframe: Play Game View (Gameplay)	39
Figure 5.1 — Sample Screenshot / Output.....	46
Figure 5.2 — Sample Screenshot / Output.....	46
Figure 5.3 — Sample Screenshot / Output.....	47
Figure 5.4 — Sample Screenshot / Output.....	47
Figure 6.1 — Benchmark Chart: Planner Service Latency	56
Figure 6.2 — Benchmark Chart: Code Assembly Latency	58
Figure 6.3 — Benchmark Chart: Code Assembly Scaling	58
Figure 6.4 — Benchmark Chart: Asset Generation Throughput	60
Figure 6.5 — Benchmark Chart: End-to-End Pipeline Latency	61
Figure 6.6 — Production: Pipeline Outcome Distribution	75
Figure 6.7 — Production: Per-Stage Mean Latency (log scale)	63
Figure 6.8 — Production: End-to-End Latency per Terminal Run	64

Figure 6.9 — Production: Pipeline Failure Causes.....	65
Figure 6.10 — Production: Deliverable Artifacts by Total Storage	66
Figure 6.11 — Production: AI-Generated Assets by Type.....	67
Figure 6.12 — Production: MinIO Object Storage by Content Type	82
Figure 6.13 — Production: Container Memory Footprint (idle)	83

List of Tables

Table 1.1 — Tools and Technologies.....	4
Table 2.1 — Comparative Analysis of Related Systems	10
Table 3.1 — Stakeholder Analysis	11
Table 3.2 — Use Case Descriptions	17
Table 4.1 — Service Catalogue	20
Table 5.1 — Implementation Environment	42
Table 6.1 — Testing Scope by Service	50
Table 6.2 — Mocking and Isolation Strategy	51
Table 6.3 — Test Suite Summary	53
Table 6.4 — Code Coverage by Service	54
Table 6.5 — Planner Service: Latency Measurements	55
Table 6.6 — Planner Service: Plan Quality Scores	56
Table 6.7 — Code Assembly: Latency vs. Complexity	57
Table 6.8 — Asset Generation: Output Breakdown	59
Table 6.9 — End-to-End Pipeline: Stage Breakdown.....	61
Table 6.10 — Production Deployment: Executive Summary	62
Table 6.11 — Pipeline Outcome Distribution (n = 33)	62
Table 6.12 — Per-Stage Latency, Completed Executions (seconds).....	63
Table 6.13 — End-to-End Latency by Outcome (seconds)	64
Table 6.14 — Pipeline Failure Causes (all 7 in the assets stage)	65
Table 6.15 — Deliverable Bundle Composition (13 builds, 78 artifacts).....	65
Table 6.16 — AI-Generated Assets by Type / Entity (n = 154)	66
Table 6.17 — MongoDB Collection Statistics	80
Table 6.18 — MinIO Object Storage by Content Type	81
Table 6.19 — MinIO Storage by Area	82

Table 6.20 — Container Resource Utilization (idle steady-state) 83

Table 6.21 — Estimated (Ch. 6.3) vs. Measured (Production) Performance 68

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
COEP	Cross-Origin Embedder Policy
COOP	Cross-Origin Opener Policy
CLI	Command-Line Interface
DXA	Document eXtensible Architecture (Word measurement unit)
ERD	Entity-Relationship Diagram
ES	ECMAScript
HTML5	HyperText Markup Language version 5
HTTP	HyperText Transfer Protocol
iframe	Inline Frame
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCP	Model Context Protocol
MinIO	Minimal Input/Output (S3-compatible object storage)
NFR	Non-Functional Requirement
PCG	Procedural Content Generation
REST	Representational State Transfer
S3	Amazon Simple Storage Service (protocol)
SPA	Single-Page Application
SQL	Structured Query Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
WASM	WebAssembly
XSS	Cross-Site Scripting
ZIP	Zone Information Protocol (compression format)

Chapter 1: Introduction

1.1 Background and Motivation

The global video game industry generated revenues exceeding \$180 billion in 2023, yet the barrier to entry for independent game development remains extremely high. Creating even a simple 2D game requires proficiency in a game engine, programming skills in a domain-specific scripting language, artistic ability to produce visual assets, audio production knowledge, and a substantial time investment. For aspiring developers, students, and creatives who have compelling game ideas but lack this full skill set, the gap between concept and playable prototype is discouraging.

Concurrently with this challenge, large language models (LLMs) have demonstrated remarkable capabilities in understanding natural language intent and generating structured artifacts including source code, JSON data, and creative descriptions. Image generation APIs can now produce stylistically consistent art from text prompts, and audio synthesis models can generate sound effects and music on demand. These capabilities when orchestrated intelligently present an opportunity to dramatically lower the barrier to game creation.

Game Forge was built to validate this opportunity: can a user describe a game in plain English and receive a fully playable browser game in return? The project targets Godot 4 as the game engine because it is free, open-source, actively developed, and supports HTML5 export that runs in any modern browser without installation.

1.2 Problem Definition

Transforming a high-level game idea into a playable prototype remains a manual and resource-intensive process that requires coordination across multiple development stages. Specifically:

1. High barrier to entry: Game development requires knowledge of gameplay logic, engine-specific scripting, asset pipelines, and build configuration. Individual developers, students, and small teams often lack the technical background needed to assemble these components into a functional prototype.

2. Fragmented development workflow: Core elements of a game—such as planning, code implementation, visual assets, audio assets, and execution—are typically developed and managed independently. Coordinating these components manually increases complexity and slows iteration cycles.

3. Lack of orchestration and automation: Existing solutions do not provide a standardized orchestration mechanism that ensures coherent communication between planning, asset generation, code construction, and build execution stages. This fragmentation often results in incomplete or non-functional prototypes that fail to accurately represent the intended game design.

1.3 Project Objectives and Proposed Solution

Game Forge addresses these problems through the following objectives:

- Accept a free-form natural-language game description as the sole user input.
- Automatically classify the game type and generate a machine-readable game plan using an LLM.
- Generate styled game assets (sprites, backgrounds, music, sound effects) through AI image and audio APIs.
- Assemble a complete, ready-to-run Godot 4 GDScript codebase by selecting and composing scripts from a curated library of pre-built, validated game components tailored to the game plan's requirements.
- Compile and export an HTML5 game using the Godot CLI in headless mode.
- Deliver the playable game to the user through the browser with no manual steps required.

The proposed solution is a microservice architecture in which a React frontend communicates with a REST backend that orchestrates a pipeline of specialized AI worker services — including a library-driven GDScript assembly engine and an LLM-powered asset generation agent — all containerized with Docker for reproducible deployment.

1.4 Project Scope and Limitations

The system supports three game archetypes: 2D platformer (side-view, gravity-based movement), top-down (overhead omnidirectional movement), and endless-runner (auto-scrolling with jump-and-duck controls). Within each archetype, a curated library of pre-built, validated GDScript components is maintained, covering a set of behaviors (patrol, chase, ranged_attacker, wander, jumping_enemy, flying_chase, stationary_hazard, moving_platform, etc.) and weapon types (melee, ranged). The appropriate scripts are selected and assembled automatically based on the game plan's requirements.

Current limitations include:

- The system does not support 3D games, physics simulations, or network multiplayer.
- Game archetypes are constrained to the three supported families; genres such as strategy, puzzle, or simulation are not currently supported.
- The GDScript library covers a fixed set of behaviors; game plans requiring mechanics outside the library's scope will have those behaviors omitted or substituted with the closest available alternative.
- AI-generated assets may exhibit inconsistency across entities if the Stability AI model produces stylistically divergent results.
- Build time ranges from approximately two-and-a-half to five minutes depending on asset count and API response latency.

1.5 Development Methodology

The project was developed using an iterative agile methodology with two-week sprints. Development proceeded in three main phases:

- (1) pipeline architecture and orchestration, establishing microservice topology and inter-service communication;
- (2) intelligent generation, implementing the planner, code generation pipeline, and asset MCP server;

(3) integration and hardening, adding the Godot syntax validation loop, error recovery, build polling, and frontend refinements. Services were developed and tested independently using Docker Compose, with integration tested against the full stack after each phase.

1.6 Tools and Technologies Used

Category	Technology	Version / Notes
Frontend	React [16] + Vite [17]	JSX, Tailwind CSS [18], Framer Motion [19]
Backend API	Node.js [20] + Express [21]	v20/v22, ES modules, Mongoose 8 [22]
Database	MongoDB [23]	Mongoose ODM, via mongo:latest Docker image
Object Storage	MinIO [24]	S3-compatible, stores scripts/assets/exports
LLM Provider	Groq API [25] / OpenRouter [26]	z.ai/glm5.2 [27], llama-3.3-70b [28], qwen3-32b [29], gpt-oss-120b [30], deepseek-v4-pro [31]
Image Generation	Stability AI [32]	stable-image-core model
Audio Generation	Stable Audio 2 [33]	stable-audio-2 model
MCP Server	FastMCP [34] (Python)	Python 3.13, Streamable HTTP transport
Game Engine	Godot 4 [35]	v4.3, headless CLI, HTML5 export
Containerization	Docker [36] + Compose [37] + GitHub Container Registry [38]	10 containers, bridge network
CI/CD	GitHub Actions [39]	Docker image publish workflows
Code Quality	ESLint [40]	Per-service analysis
Auth	JWT [41] + bcryptjs [42]	Bearer tokens, password hashing
Security	Helmet [43], rate-limit [44], xss-clean [45], mongo-sanitize [46]	Express middleware stack

Table 1.1 — Tools and Technologies

1.7 Project Timeline / Gantt Chart

The project was executed over two semesters (approximately 30 weeks). The chart below summarizes the major milestone periods.

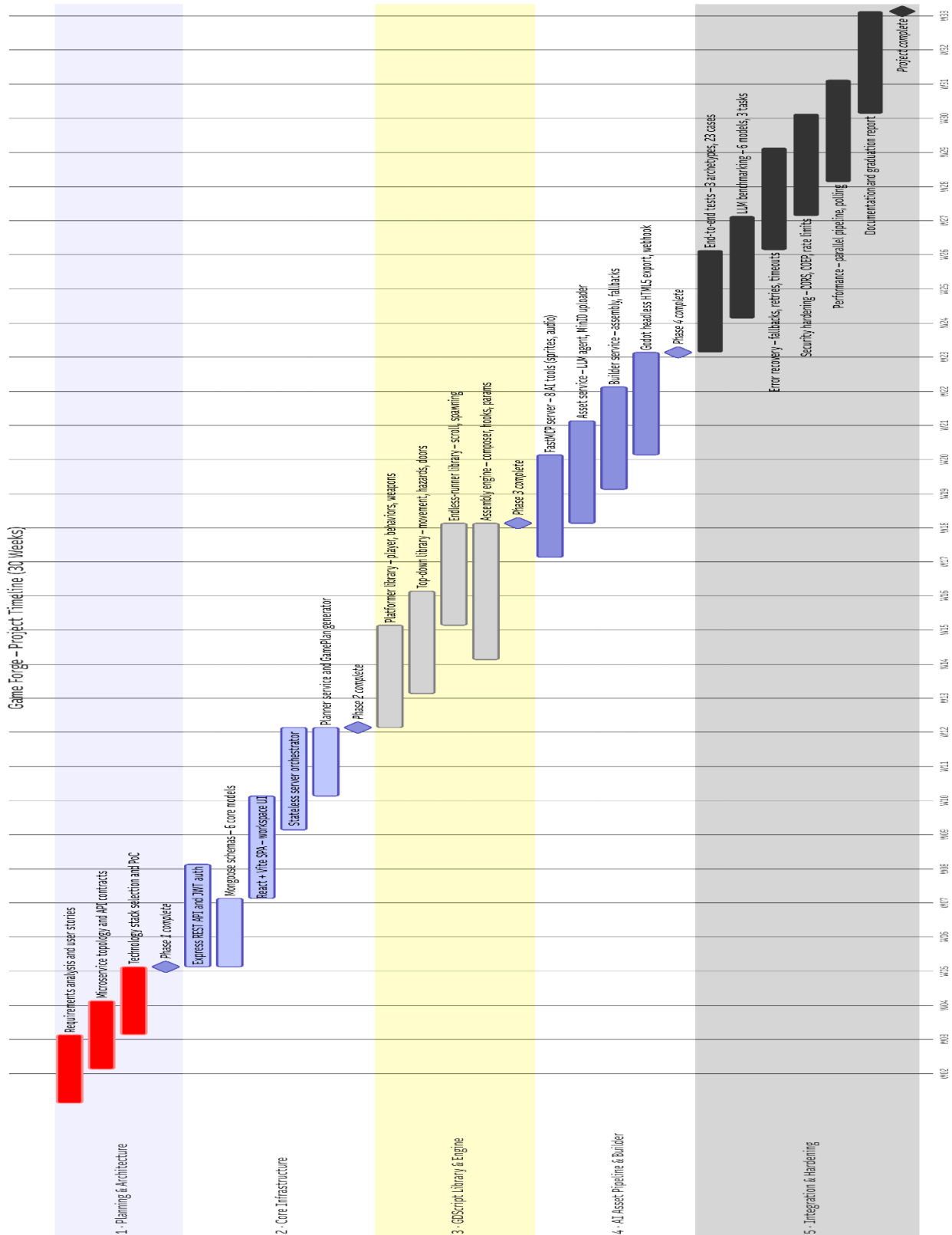


Figure 1.1

1.8 Report Organization

The remainder of this document is organized as follows.

Chapter 2 surveys related work in AI-assisted game generation and automated code generation.

Chapter 3 presents the system analysis including stakeholder analysis, functional and non-functional requirements, use case diagrams, and a feasibility study.

Chapter 4 describes the system design through architecture diagrams, class diagrams, sequence diagrams and the entity-relationship model.

Chapter 5 details the implementation environment, key algorithms, and sample outputs.

Chapter 6 covers the testing strategy, test cases, benchmark results, and known limitations.

Chapter 7 concludes with a summary of achievements, lessons learned, and future work directions.

Chapter 2: Related Work

The automated and AI-assisted game creation space has grown rapidly, but existing solutions cluster into distinct categories that each address only part of the idea-to-playable-game problem.

2.1 Prompt-to-Game Platforms

This category contains the systems most directly comparable to Game Forge: platforms that accept a natural-language description and return a playable game.

2.1.1 Rosebud AI

Rosebud AI [1] is a browser-based, no-code platform that generates playable 2D and 3D games from natural-language prompts through a workflow it calls "vibe coding." The user describes a game and refines it via chat; the platform generates the game's code, AI visual assets, and sprite-sheet animations, running the result in its own JavaScript-based web runtime rather than a standalone engine [1]. It uses LLMs for code generation alongside generative image models for art. Unlike Game Forge, which emits real GDScript in a portable Godot 4 project, Rosebud's output is bound to its proprietary runtime.

2.1.2 Playabl.ai

Playabl.ai (Y Combinator, 2026) [2] converts text descriptions into playable browser games surfaced through a TikTok-style play-and-remix feed. Rather than generating a whole game in one LLM pass, its agent first turns the idea into a structured game design, then its engine programmatically translates that design into reusable components (movement, enemies, scoring, levels, UI), which the agent finally connects into a playable game [2]. It is built on a proprietary AI-native engine with native 3D asset generation and a preset animation library [3]. This design-then-assemble-components model is the closest analog to Game Forge's own library-assembly approach, though Game Forge targets open-source Godot 4 instead.

2.1.3 Astrocade

Astrocade [4] is an AI-powered social gaming platform built on "wish-to-game" technology that creates casual games from prompts, complete with AI-generated art, sound, and music. It uses a team of specialized AI agents — separate agents for

visuals, UI, and gameplay feel — that collaborate to build the game and even generates a partly custom editor per game for iteration [5]. Built on a proprietary engine, it is oriented toward social discovery, remixing, and creator monetization [6]. Its multi-agent split mirrors Game Forge's microservice decomposition, but Astrocade targets a closed social ecosystem rather than producing a portable, downloadable game.

2.1.4 StoneLeap

StoneLeap [7] is an AI-first platform combining prompt-based game creation, a drag-and-drop visual editor, cloud gaming infrastructure, and built-in monetization. It applies AI across the full stack — assets, code, game logic, and vision — and supports native export to Steam, the App Store, and other targets, positioning itself as an all-in-one "create-and-monetize" ecosystem [7]. It is built on a proprietary engine and commercial cloud backend. Like Game Forge it spans assets through code, but its closed, monetization-centric platform contrasts with Game Forge's open-source Godot 4 engine and transparent library-driven generation.

2.2 AI Asset and Design Tools

The systems in this category address parts of the game-creation pipeline — asset creation or design ideation — without producing a complete playable game in the manner of Game Forge.

2.2.1 Ludo.ai

Ludo.ai [8] is an AI game-design and asset platform offering market research across millions of titles, guided brainstorming, and generators for sprites, animations, icons, UI, textures, music, 3D models, and video. Its generators are exposed through an MCP server, REST API, and a Unity plugin, so assets can be produced on demand from tools like Claude or Cursor; a separate Playable Generator turns text into a simple JavaScript web prototype [8], [9]. Notably, both Ludo and Game Forge expose asset generation over the Model Context Protocol [10]. The difference is scope: Ludo is an asset pipeline for developers to use themselves, whereas asset generation in Game Forge is one automated stage of a complete code-and-build pipeline.

2.2.2 GDevelop AI

GDevelop [11] is a mature open-source 2D/3D engine whose logic is built from a visual event system of conditions and actions rather than code. Its AI comes as an "Ask AI" chat assistant and a "Build for me" agent that reads the current project and directly creates or modifies objects, events, and behaviors from plain-language instructions, grounded in the engine's official library of behaviors and extensions [12]. It is built on general-purpose LLMs adapted to GDevelop's event model [13]. Like Game Forge it pairs an open-source HTML5 engine with a defined component library, but GDevelop's AI is an in-editor assistant building features incrementally, not an automated one-prompt pipeline that compiles a whole game [12].

2.3 General-Purpose Code Generation

2.3.1 LLM Code Assistants (GitHub Copilot and Similar)

GitHub Copilot [14] and the broader class of code-generating LLMs produce game-logic snippets from comments or prompts and can assist scripting in engines like Unity and Godot. They are built on large general-purpose code models integrated into the developer's editor [14]. Their limitation is that they emit isolated snippets, not complete integrated projects: they don't understand a game's architecture, generate no assets, assemble no project structure, and compile no build. Game Forge instead produces an entire, architecturally consistent Godot project and compiles it, using LLMs only in bounded roles (classification, plan generation, asset prompts) while assembling the code deterministically from a validated library.

2.3.2 Procedural Content Generation Systems

Procedural Content Generation (PCG) is the algorithmic creation of game content — levels, maps, dungeons, items — through rule-based, constraint-based, and search-based techniques [15]. PCG is deterministic, fast, and widely used in commercial games to generate content variations within a fixed, hand-authored framework [15]. Its methods are algorithms, not learned models. PCG shares Game Forge's philosophy of producing content from structured rules rather than free generation, but classical PCG only varies content inside one pre-built game; it cannot turn a free-form description into an entirely new game with bespoke assets and logic, as Game Forge does.

2.4 Comparative Analysis

The table below summarizes how each surveyed system compares to Game Forge.

System	Natural-Language Input	Output	Asset Generation	Engine / Runtime	Code Generation Approach	Open Source
Game Forge (ours)	Free-form text + chat	Complete playable HTML5 game (downloadable Godot project)	AI-generated sprites, backgrounds, music, SFX (Stability AI + Stable Audio via MCP)	Godot 4 (open-source)	Deterministic assembly from curated, validated GDScript library	Yes
Rosebud AI	Free-form text + chat	Playable 2D/3D web game	AI-generated visuals + sprite animations	Proprietary web runtime	LLM code generation (JavaScript)	No
Playabl.ai	Text + guided questions	Playable browser game (remixable)	Native 3D assets + animation library	Proprietary AI-native engine	Structured design → modular component assembly	No (code viewable)
Astrocade	Free-form text	Playable casual social game	AI-generated art, sound, music	Proprietary engine	Multi-agent generation	No
Ludo.ai	Text (per generator)	Assets + simple playable prototype	Sprites, audio, 3D, video, UI (via MCP/API)	Engine-agnostic assets; web prototype	LLM prototype generation (JS)	No
StoneLeap	Free-form text	Published game (multi-platform export)	Full-stack AI assets	Proprietary engine	AI across full stack	No
GDevelop AI	Text instructions	Editable game features inside editor	User-provided / library assets	GDevelop (open-source)	AI agent builds visual events from a behavior library	Yes
LLM Assistants (Copilot etc.)	Code comments / prompts	Code snippets	None	Any (assistive)	Free-form LLM code generation	No
PCG Systems	Seeds / parameters	Levels / content variation	Procedural / none	Various	Rule/search-based algorithms	Varies

Table 2.1 — Comparative Analysis of Related Systems

Chapter 3: System Analysis

3.1 Stakeholder Analysis

Stakeholder	Role	Interests / Expectations
End User (Game Creator)	Primary user of the platform	Describe a game and receive a playable result with minimal technical knowledge required
Game Forge Team	Developers and maintainers	Reliable pipeline, extensible architecture, well-tested services
Supervisor / Academic Committee	Evaluators	Technical rigor, novelty, completeness of documentation and implementation
AI API Providers (Groq, OpenRouter, Stability AI)	External services	Correct API usage, rate limit compliance, API key security
Godot Engine Community	Open-source engine	Correct GDScript usage, valid project file format compliance

Table 3.1 — Stakeholder Analysis

3.2 Functional Requirements

3.2.1 User Authentication and Account Management

- **FR-1.1** — The system shall allow users to create an account using a valid email address and password.
- **FR-1.2** — The system shall authenticate users before granting access to platform features.
- **FR-1.3** — The system shall allow authenticated users to log out securely.
- **FR-1.4** — The system shall allow authenticated users to update their profile information and change their password.

3.2.2 User Profile and Project Management

- **FR-2.1** — The system shall allow users to view their profile information.
- **FR-2.2** — The system shall allow users to view a list of their generated game projects.
- **FR-2.3** — The system shall allow users to view detailed information about each project.

- **FR-2.4** — The system shall allow users to manage their generated projects.

3.2.3 Prompt Submission and Validation

- **FR-3.1** — The system shall allow users to submit natural-language descriptions of game ideas.
- **FR-3.2** — The system shall create a unique project for each submitted game prompt.
- **FR-3.3** — The system shall provide an AI-powered chat interface that allows users to refine their game design before confirming generation.

3.2.4 Game Planning

- **FR-4.1** — The system shall automatically convert user prompts into structured game design specifications.
- **FR-4.2** — The system shall store generated game plans.

3.2.5 Asset Generation and Management

- **FR-5.1** — The system shall generate visual assets required for the game.
- **FR-5.2** — The system shall generate or prepare audio assets for the game.
- **FR-5.3** — The system shall associate generated assets with their corresponding game projects.

3.2.6 Code and Scene Generation

- **FR-6.1** — The system shall assemble game scripts compatible with the Godot engine.
- **FR-6.2** — The system shall assemble game scenes and entities based on the game design plan.

3.2.7 Build and Execution

- **FR-7.1** — The system shall assemble code and generated assets into a complete game project.
- **FR-7.2** — The system shall export a playable HTML5 version of the generated game.

- **FR-7.3** — The system shall track and manage game generation execution tasks.

3.2.8 Game Access and Distribution

- **FR-8.1** — The system shall allow users to play the generated game through a web browser.
- **FR-8.2** — The system shall allow users to download the generated Godot project files.

3.2.9 Status Reporting and Error Handling

- **FR-9.1** — The system shall display execution status updates for game generation tasks.
- **FR-9.2** — The system shall notify users if a game generation task fails or completes successfully.

3.3 Non-Functional Requirements

3.3.1 Security

- **NFR-1.1** — All user passwords shall be stored using a one-way salted hash (bcrypt, cost factor ≥ 10). The system shall never persist or return plaintext passwords; the password field shall be excluded from query results by default.
- **NFR-1.2** — All authenticated API endpoints shall require a valid, unexpired JSON Web Token. Requests bearing an absent, malformed, or expired token shall be rejected with an HTTP 401 response.
- **NFR-1.3** — A token issued before a user's most recent password change shall be treated as invalid, forcing re-authentication after a credential change.
- **NFR-1.4** — All internal service-to-service endpoints shall be inaccessible to external clients and s1.dohall require a shared service credential; requests without it shall receive an HTTP 401/403 response.

- **NFR-1.5** — Authentication endpoints shall be rate-limited per client IP (≤ 20 requests per 15-minute window) to mitigate brute-force attacks, returning HTTP 429 when the limit is exceeded.
- **NFR-1.6** — All user-supplied input shall be sanitized against NoSQL injection and cross-site scripting before processing or persistence.

3.3.2 Performance

- **NFR-2.1** — Under nominal single-user load, authentication and CRUD API requests shall return a response within 500 ms for the 95th percentile of requests, excluding generation tasks.
- **NFR-2.2** — The system shall expose generation progress as discrete, observable states (planning, assets, code, building, done) with a numeric completion percentage, updated at a polling interval beginning at 3 seconds and backing off to at most 15 seconds.
- **NFR-2.3** — End-to-end game generation shall complete within 10 minutes for a typical prompt; the build stage shall enforce a configurable hard timeout to prevent indefinitely running tasks.

3.3.3 Scalability

- **NFR-3.1** — The system shall process multiple concurrent generation requests, with each request tracked as an independent, isolated job that does not block or corrupt others.
- **NFR-3.2** — Each generation stage (planning, asset generation, code assembly, build) shall be deployed as an independently scalable service, allowing horizontal replication of any single stage without modifying others.
- **NFR-3.3** — New game archetypes or generation modules shall be addable by extending the component library and service interfaces, without requiring changes to the core orchestration logic.

3.3.4 Reliability

- **NFR-4.1** — A failure in any single generation stage shall be contained to the affected job: the job shall be marked failed with a recorded error, and the platform and all other jobs shall continue operating.
- **NFR-4.2** — Where an AI-driven stage produces invalid output (e.g. an unparseable or schema-invalid game plan), the system shall apply a deterministic fallback to ensure a valid, playable result is still produced.
- **NFR-4.3** — Every generation job shall terminate in an explicit, persisted terminal state (completed or failed); no job shall remain indefinitely in an in-progress state once its execution has ended.

3.3.5 Availability

- **NFR-5.1** — Each service shall expose a health-check endpoint enabling automated liveness monitoring and container restart on failure.
- **NFR-5.2** — Generated game artifacts (scripts, assets, and HTML5 builds) shall be persisted in durable object storage so that a completed game remains retrievable independent of the runtime state of the generation services.

3.3.6 Maintainability

- **NFR-6.1** — The system shall follow a microservice architecture in which each service has a single, well-defined responsibility and communicates through documented interfaces, allowing independent development, testing, and deployment.
- **NFR-6.2** — Each service shall emit structured, timestamped logs of significant events and errors, and generation jobs shall retain a persisted log trail sufficient to diagnose failures.
- **NFR-6.3** — The codebase shall maintain a consistent module standard across services and shall be analyzable by automated static-analysis tooling in the continuous-integration pipeline.

3.3.7 Usability

- **NFR-7.1** — A user shall be able to initiate game generation from an authenticated state in no more than four interactions (prompt entry, follow up questions, audio details and confirmation).
- **NFR-7.2** — The interface shall continuously communicate the current generation state to the user, distinguishing in-progress, completed, and failed outcomes without requiring a manual page refresh.
- **NFR-7.3** — On failure, the system shall present a human-readable message indicating that generation failed, rather than exposing raw error traces or internal identifiers.

3.3.8 Portability

- **NFR-8.1** — Generated games shall be exported as a self-contained HTML5 build that runs in current Chromium-based and Firefox browsers without plugins or local installation.
- **NFR-8.2** — The system shall produce a standard, portable Godot 4 project that users may download and open directly in the Godot editor.
- **NFR-8.3** — The entire platform shall be deployable on any Docker-capable host through a single orchestrated configuration, without host-specific manual setup.

3.4 Use Case Diagrams

The following usecases describe the primary interactions between actors and the system. Two actor categories are identified: the End User (authenticated human user) and the System (automated pipeline services acting on behalf of the user).

Use Case ID	Use Case Name	Actor	Description
UC-01	Register Account	End User	User provides name, email, and password; system creates an account and returns a JWT token.

Use Case ID	Use Case Name	Actor	Description
UC-02	Log In	End User	User provides credentials; system verifies the password and returns a JWT token. Includes Verify Password; extended by Display Login Error on failure.
UC-03	Create New Project	End User	User enters a game description prompt; system creates a ChatProject and starts an AI design conversation.
UC-04	Refine Game Design via Chat	End User	User exchanges messages with the AI assistant to refine game specifications before confirming.
UC-05	Confirm and Generate Game	End User	User clicks Confirm; system initiates the full generation pipeline asynchronously.
UC-06	Track Build Progress	End User / System	System polls the backend at intervals; UI displays current pipeline step and progress percentage.
UC-07	Play Game in Browser	End User	User clicks Play; system loads the HTML5 export in an iframe via the play endpoint.
UC-08	Download Game ZIP	End User	User downloads a ZIP archive of the HTML5 export from MinIO.
UC-09	Delete Project	End User	User deletes a project and its associated data.
UC-10	Update Profile / Password	End User	User updates display name or changes their password through the profile page.
UC-11	Generate Game Plan	System (Planner)	Planner classifies prompt, calls LLM, validates and persists the GamePlan, falls back to default on failure.
UC-12	Generate Game Assets	System (Asset)	Asset service orchestrates MCP agent to call Stability AI and Stable Audio for all plan entities.
UC-13	Assemble GDScript Code	System (Code)	Code service loads archetype template, composes behaviors/weapons from the library, validates with Godot, uploads scripts to MinIO.
UC-14	Build and Export Game	System (Builder)	Builder downloads scripts and assets from MinIO, runs Godot export, uploads result, notifies backend.
UC-15	Logout	End User	User ends their authenticated session securely.

Table 3.2 — Use Case Descriptions

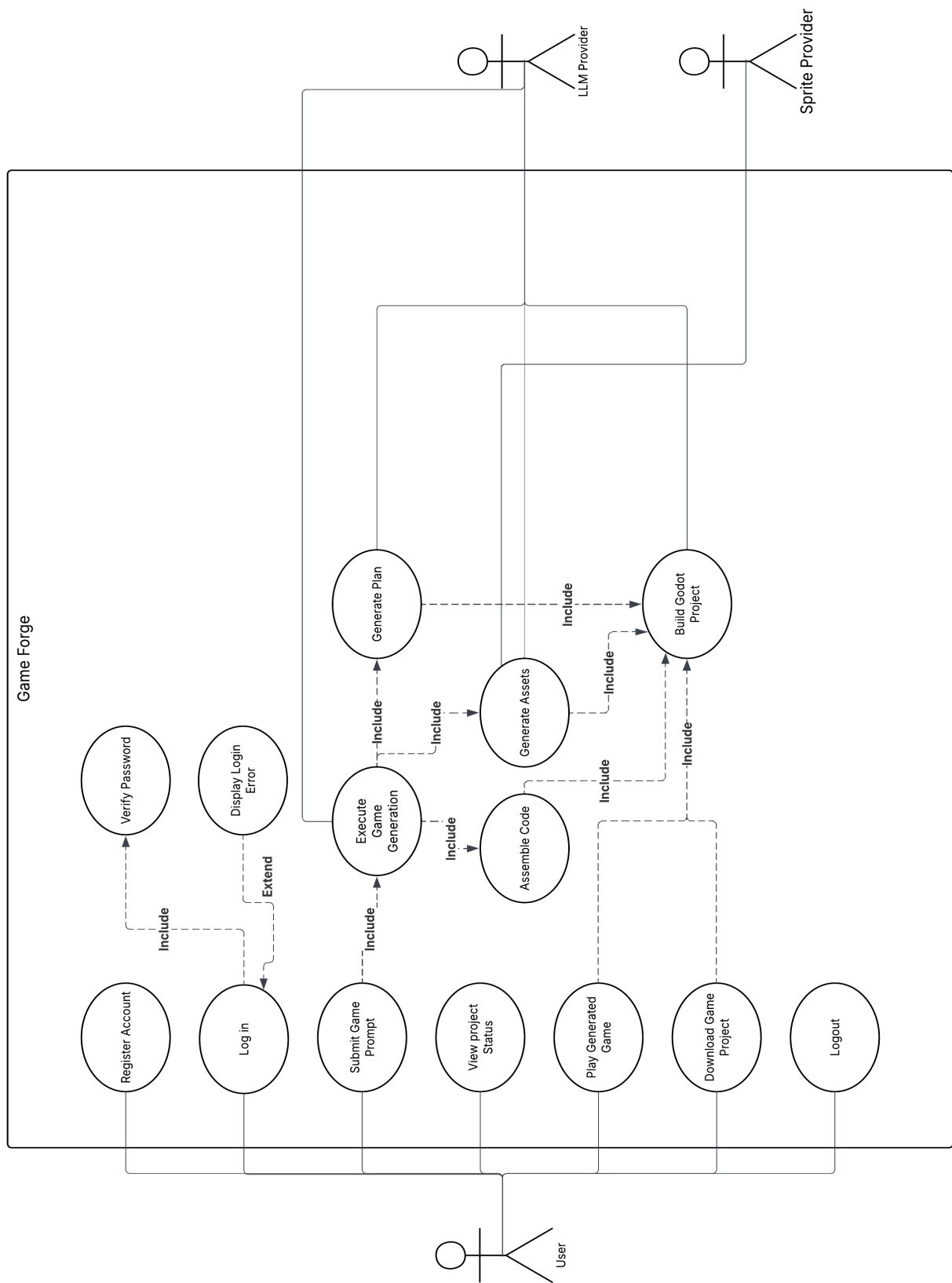


Figure 3.1 — Use Case Diagram

3.5 Feasibility Study

3.5.1 Technical Feasibility

All core technologies are mature, open-source, and actively maintained: Node.js and Express underpin the REST and orchestration layers; MongoDB (with Mongoose) and the S3-compatible MinIO handle persistence and object storage; Godot 4's headless export mode enables automated, CLI-driven HTML5 builds with programmatic validation via `--check-only`; and the Groq/OpenRouter, Stability AI, and Stable Audio APIs offer documented programmatic access, with MCP providing standard transport for the asset-generation agent. This deterministic assembly of a curated GDScript component library, with the LLM confined to bounded tasks such as classification and plan generation, avoids the reliability problems of free-form code generation, making correct, compilable output technically achievable.

3.5.2 Economic Feasibility

The platform relies almost entirely on the free and open-source technologies described above, so licensing carries no cost, and the project ran on student-owned hardware with no dedicated server or hosting expenditure. The only direct expenses were paid API subscriptions for the two external generation services — OpenRouter for LLM inference and Stability AI for image and audio generation. Based on observed usage, the total external API cost per complete game generation is estimated at under one US dollar, making the platform economically viable under a freemium or pay-per-generation model at production scale.

3.5.3 Operational Feasibility

The entire platform deploys through a single Docker Compose configuration, starting the full ten-service stack on any Docker-capable machine without host-specific setup. Its microservice decomposition lets each stage be updated, scaled, or replaced independently — for example, swapping the planner's LLM model requires no change elsewhere. Each service exposes a health-check endpoint for automated monitoring and restart, and generated artifacts persist in durable object storage independently of the generation services' runtime state. The games run directly in any modern browser via Godot's WebAssembly export, with no installation or plug-ins required.

Chapter 4: System Design

4.1 System Architecture

Game Forge follows a microservice architecture pattern with ten containerized services communicating over a private Docker bridge network. The architecture is organized into three tiers: the presentation tier (frontend), the application tier (backend, server, and four worker services), and the data/storage tier (MongoDB and MinIO).

Service	Port	Technology	Responsibility
frontend	3000	React + Vite	Single-page application: workspace UI, chat, build progress, game player
backend	5000	Node.js/Express	REST API, JWT auth, MongoDB models, proxies MinIO for game serving
server	6100	Node.js/Express	Stateless orchestrator: receives generate request, runs pipeline, pushes logs to backend
planner	6101	Node.js/Express	LLM prompt classifier + GamePlan generator; persists plan to MongoDB
asset	6102	Node.js/Express	MCP client: orchestrates assets-mcp via LLM agent, saves Asset docs
code	6103	Node.js/Express	Template-guided GDScript assembler + Godot syntax validator + MinIO uploader
builder	6104	Node.js/Express	Downloads artifacts from MinIO, runs Godot headless export, uploads HTML5
assets-mcp	8765	Python/FastMCP	MCP server: wraps Stability AI and Stable Audio APIs as MCP tools
mongo	27017	MongoDB	Persistent store for Users, GameProjects, GenerationJobs, Builds, Assets, GamePlans
minio	9000/9001	MinIO	S3-compatible object storage for GDScripts, asset images/audio, HTML5 exports

Table 4.1 — Service Catalogue

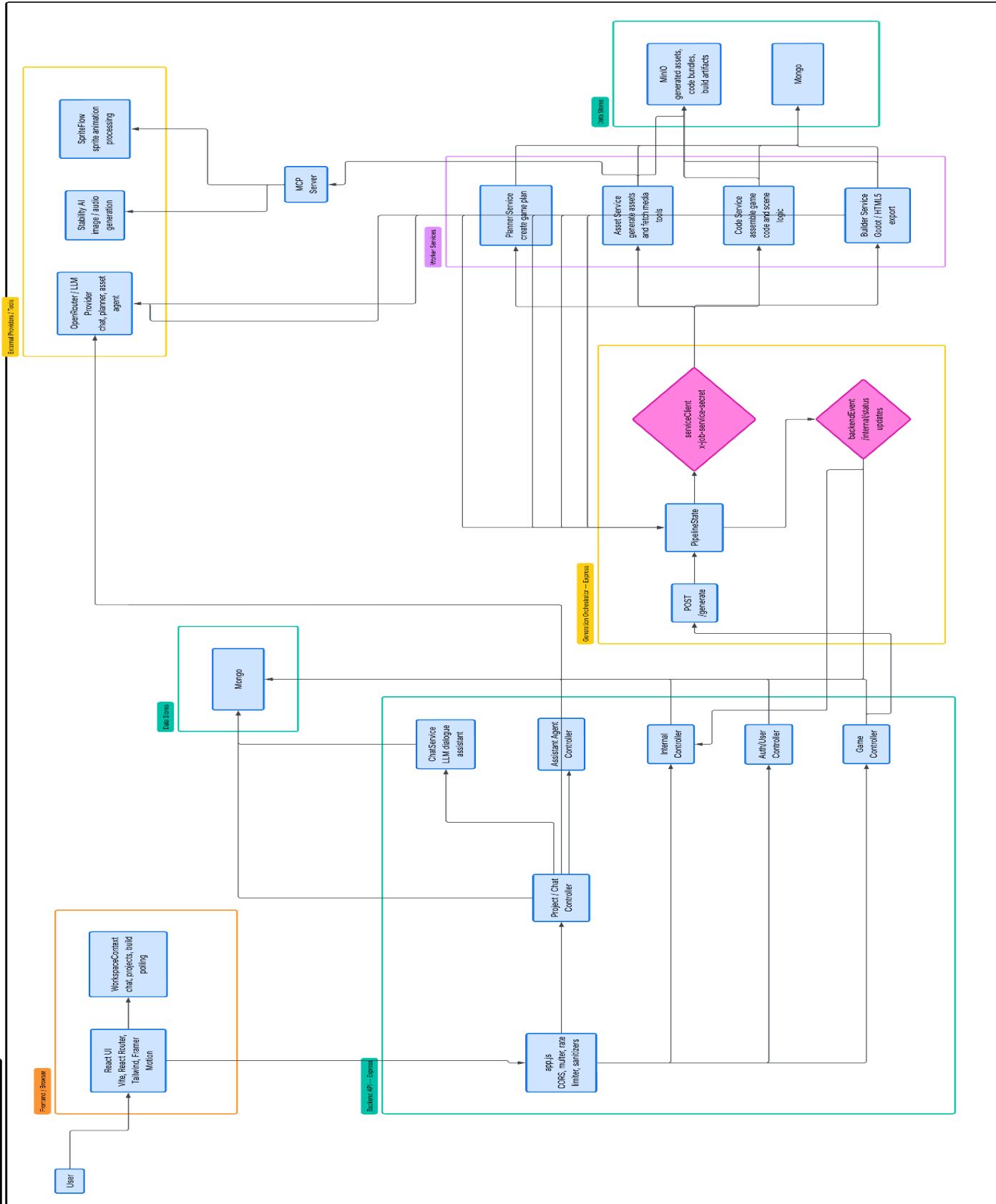


Figure 4.1 — System Architecture / Component Diagram

4.2 Class Diagrams

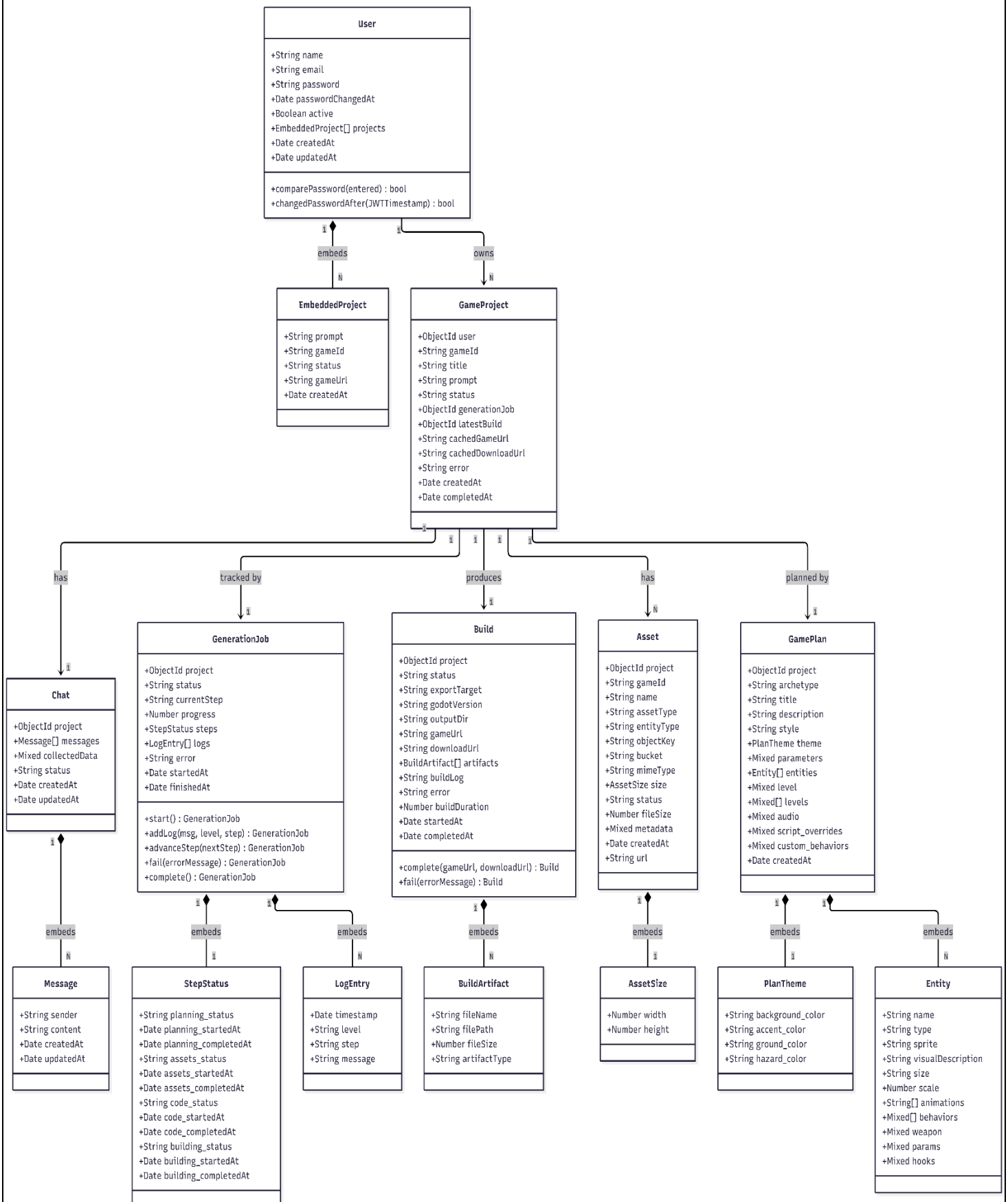


Figure 4.2 — Class Diagram: Core MongoDB Models

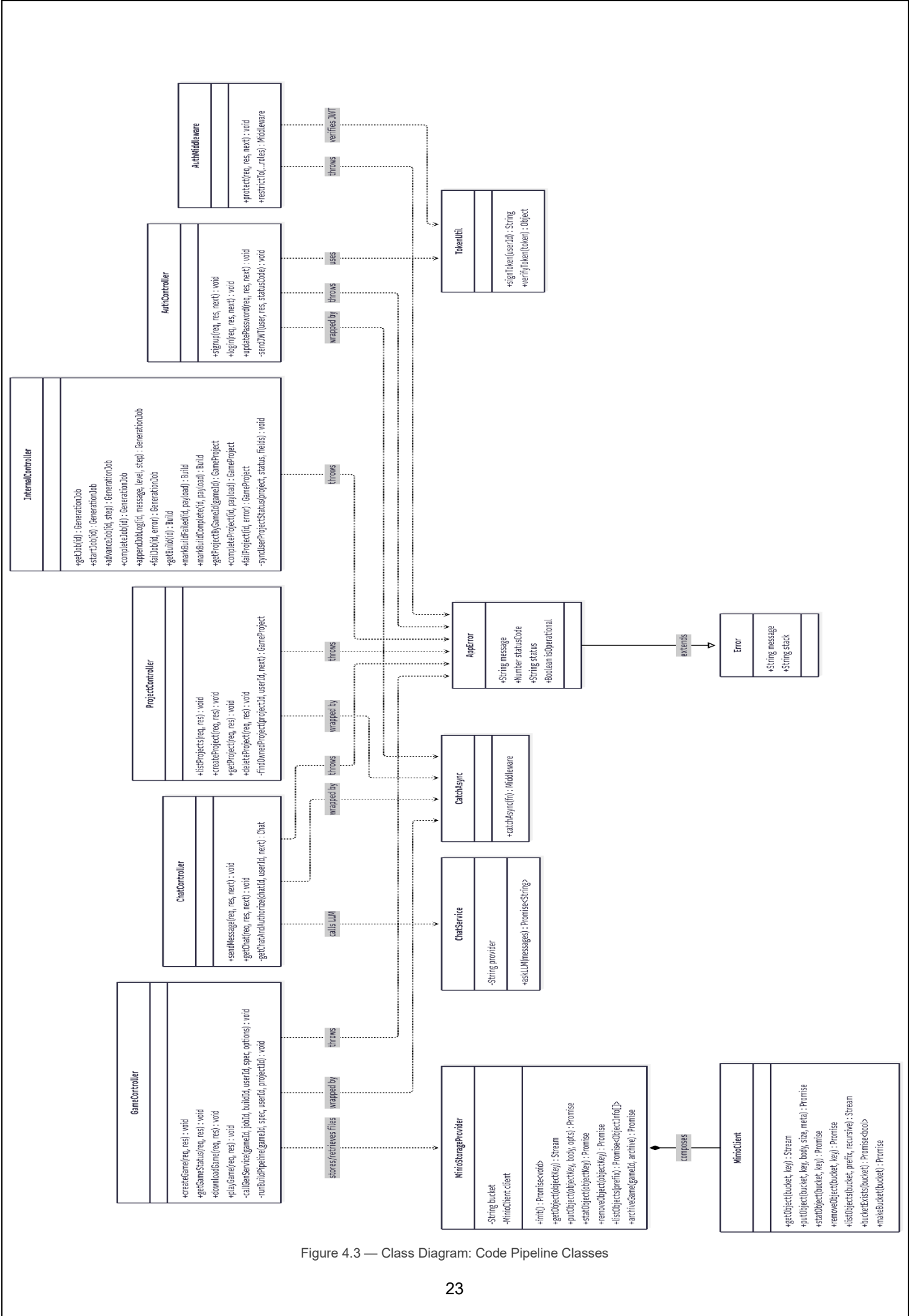


Figure 4.3 — Class Diagram: Code Pipeline Classes



Figure 4.4 — Class Diagram: Asset MCP Server Classes

4.3 Sequence Diagrams

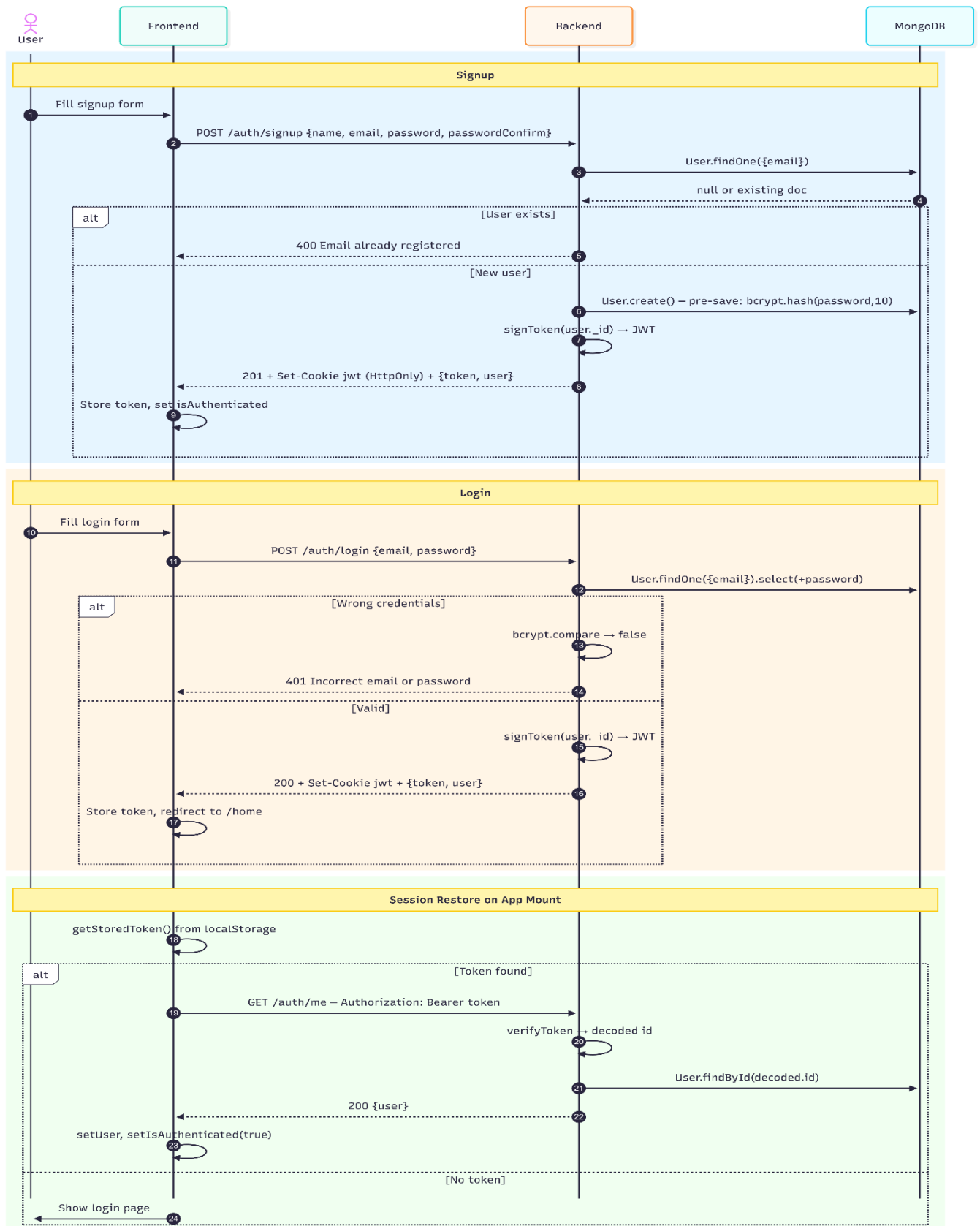


Figure 4.6 — Sequence Diagram: User Registration and Login

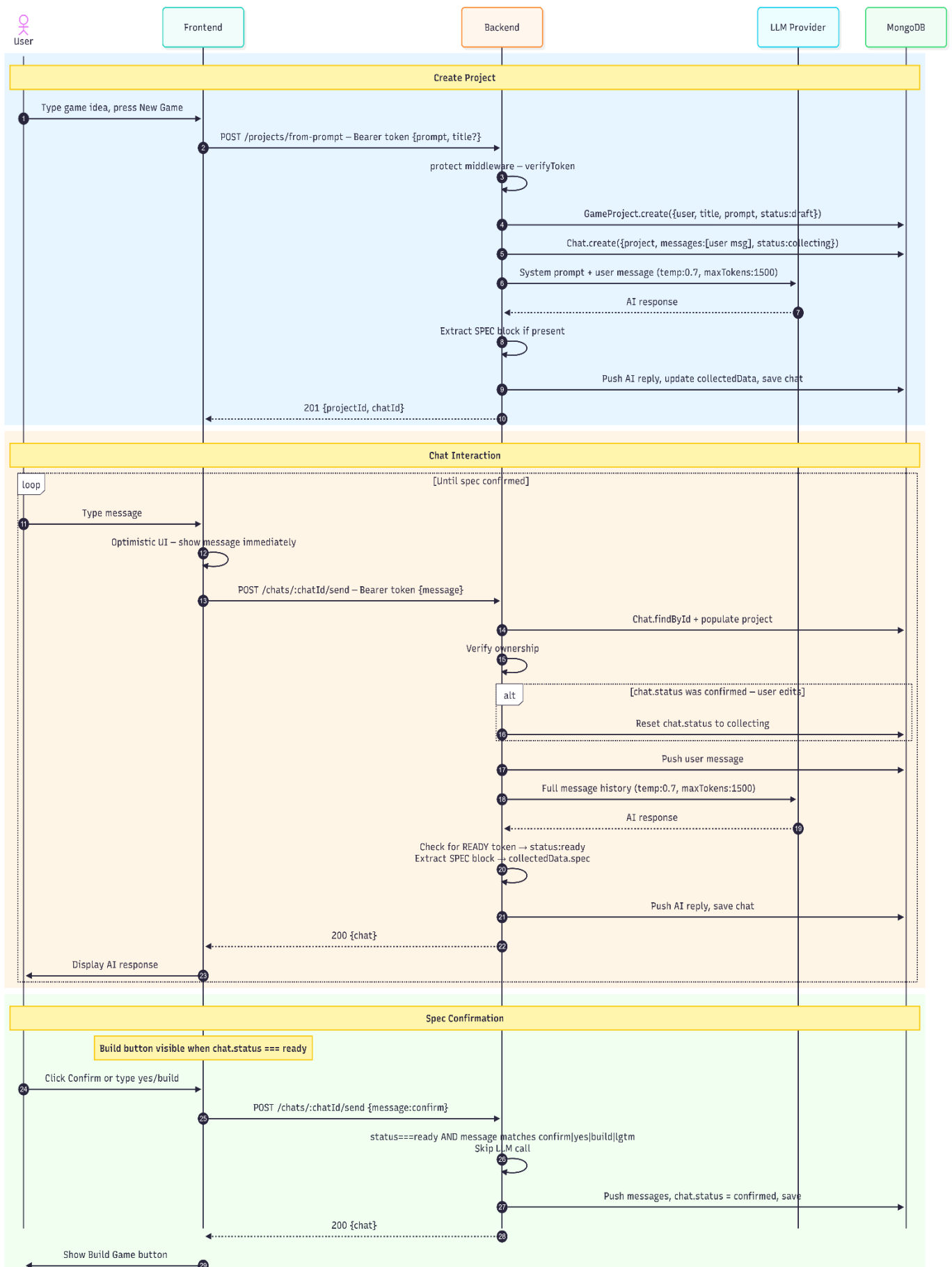


Figure 4.7 — Sequence Diagram: Create Project and Chat Refinement

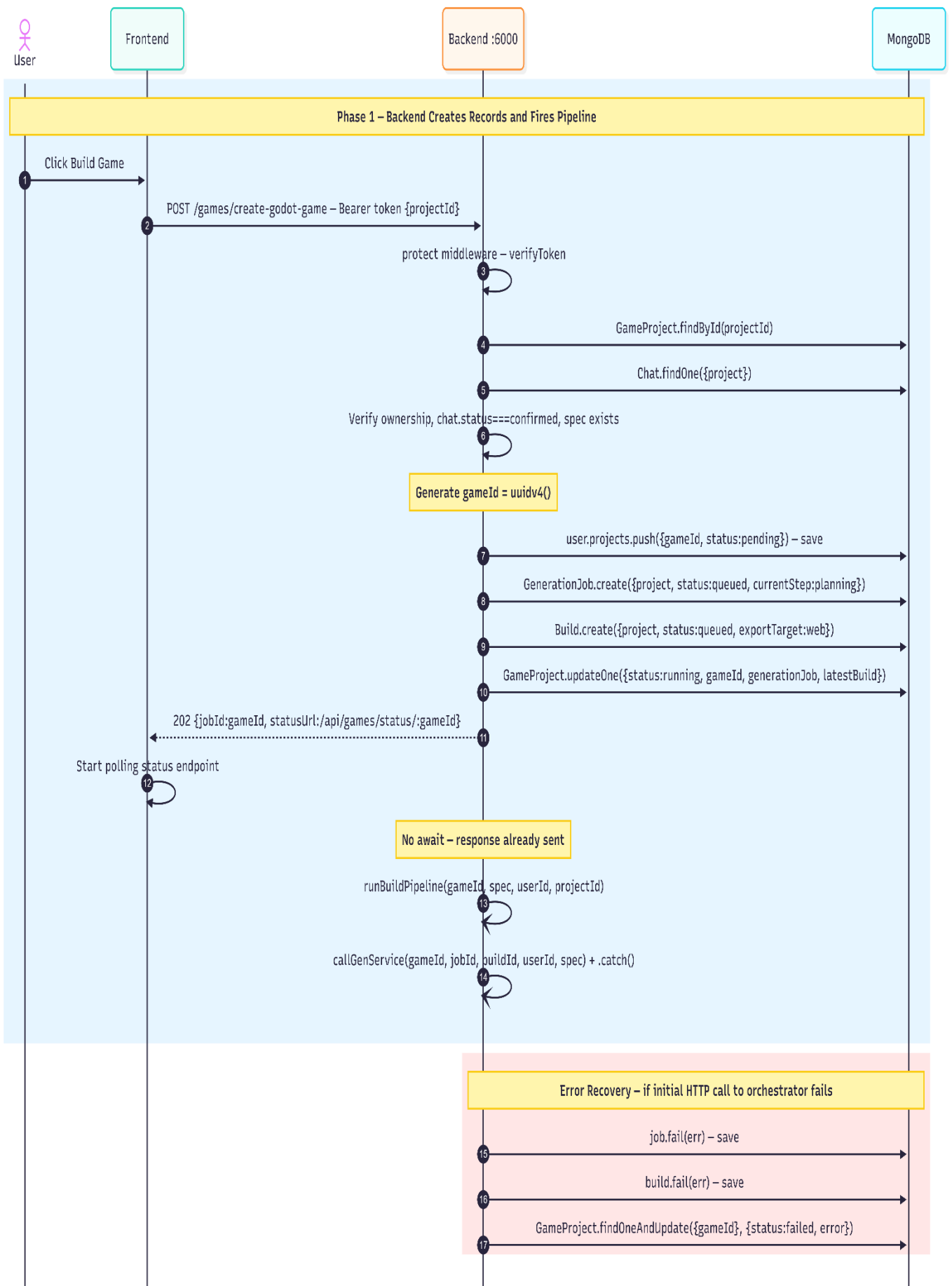


Figure 4.8 — Sequence Diagram: Full Game Generation Pipeline

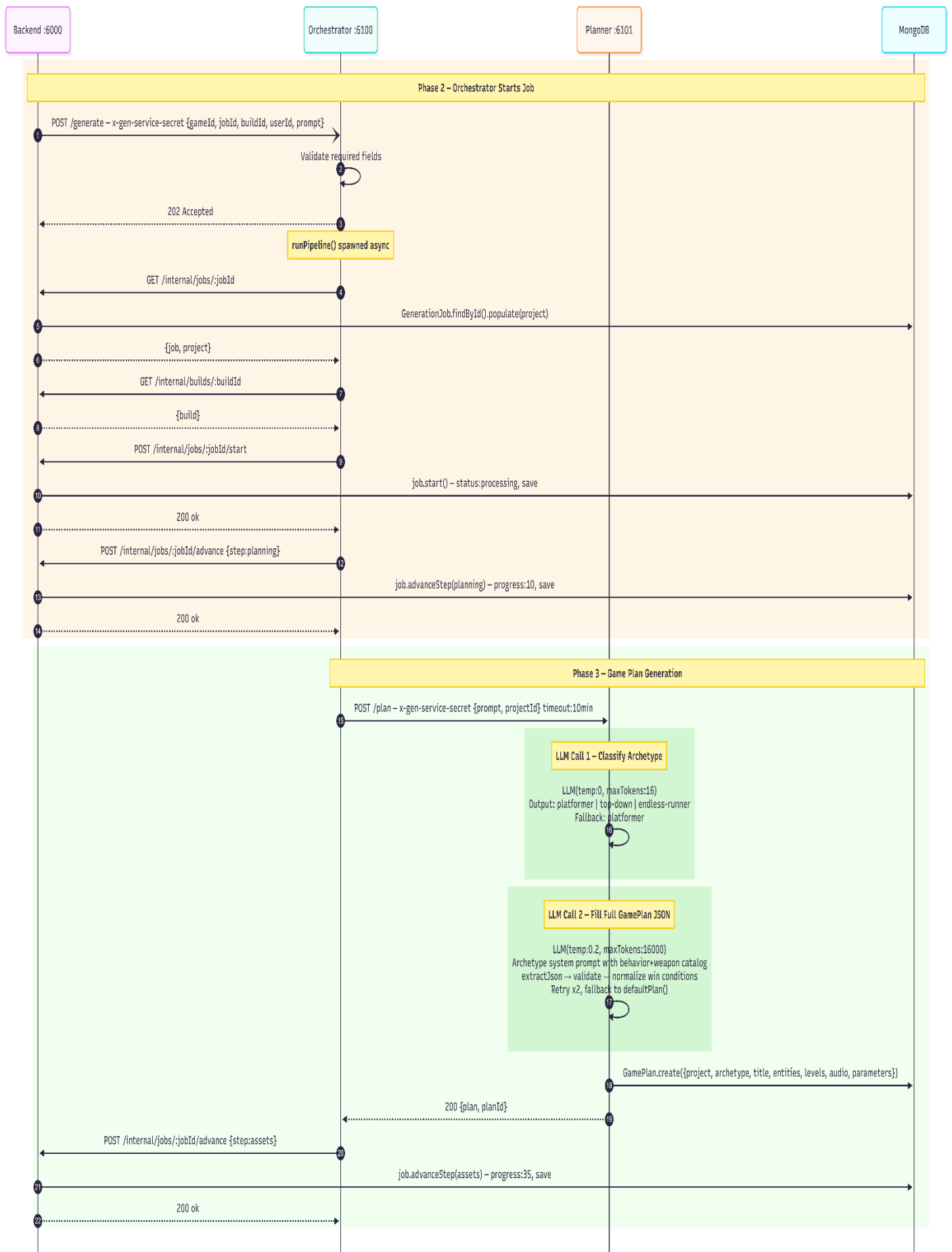


Figure 4.9 — Sequence Diagram: Asset Generation via MCP

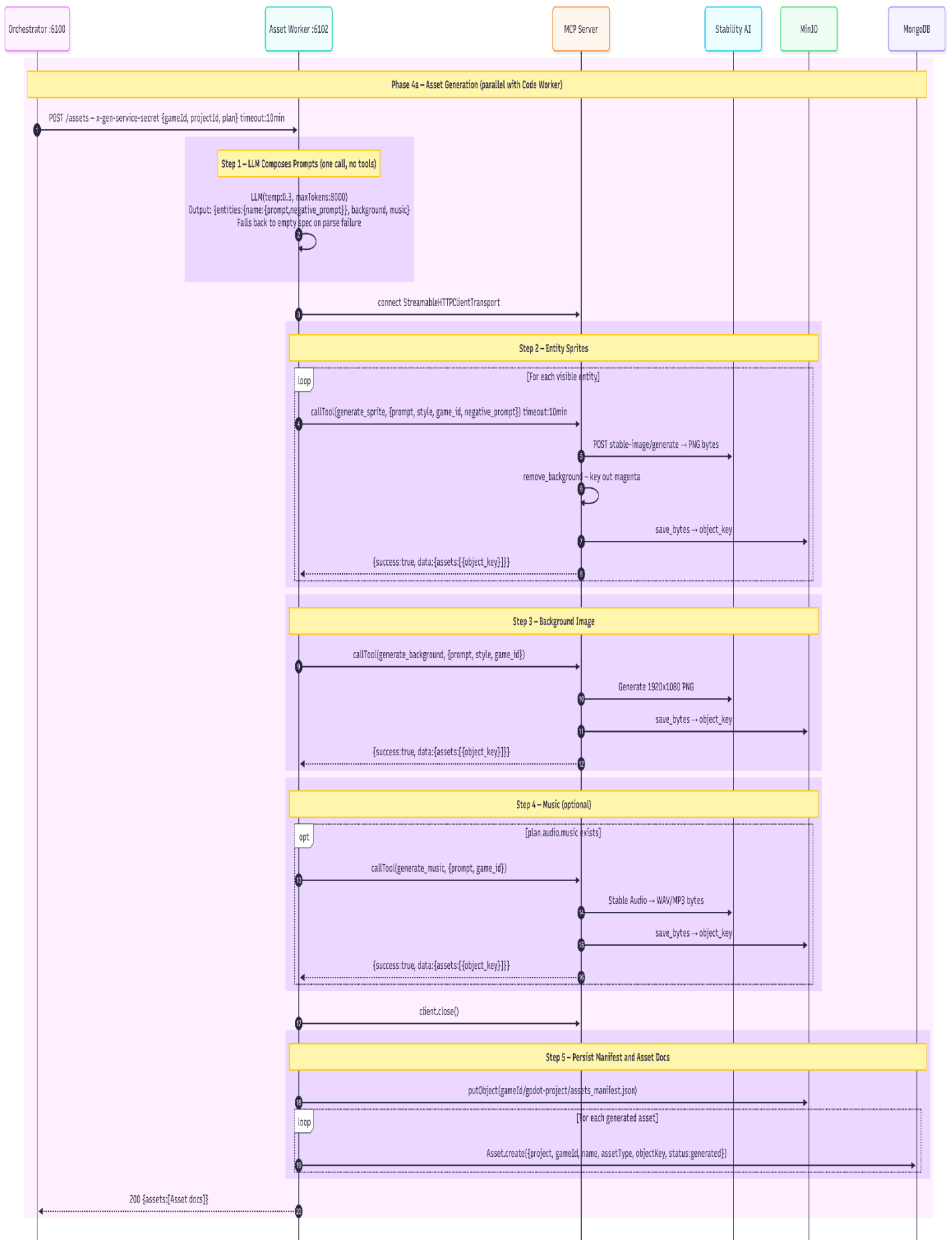


Figure 4.10 — Sequence Diagram: Code Generation with Validation Loop

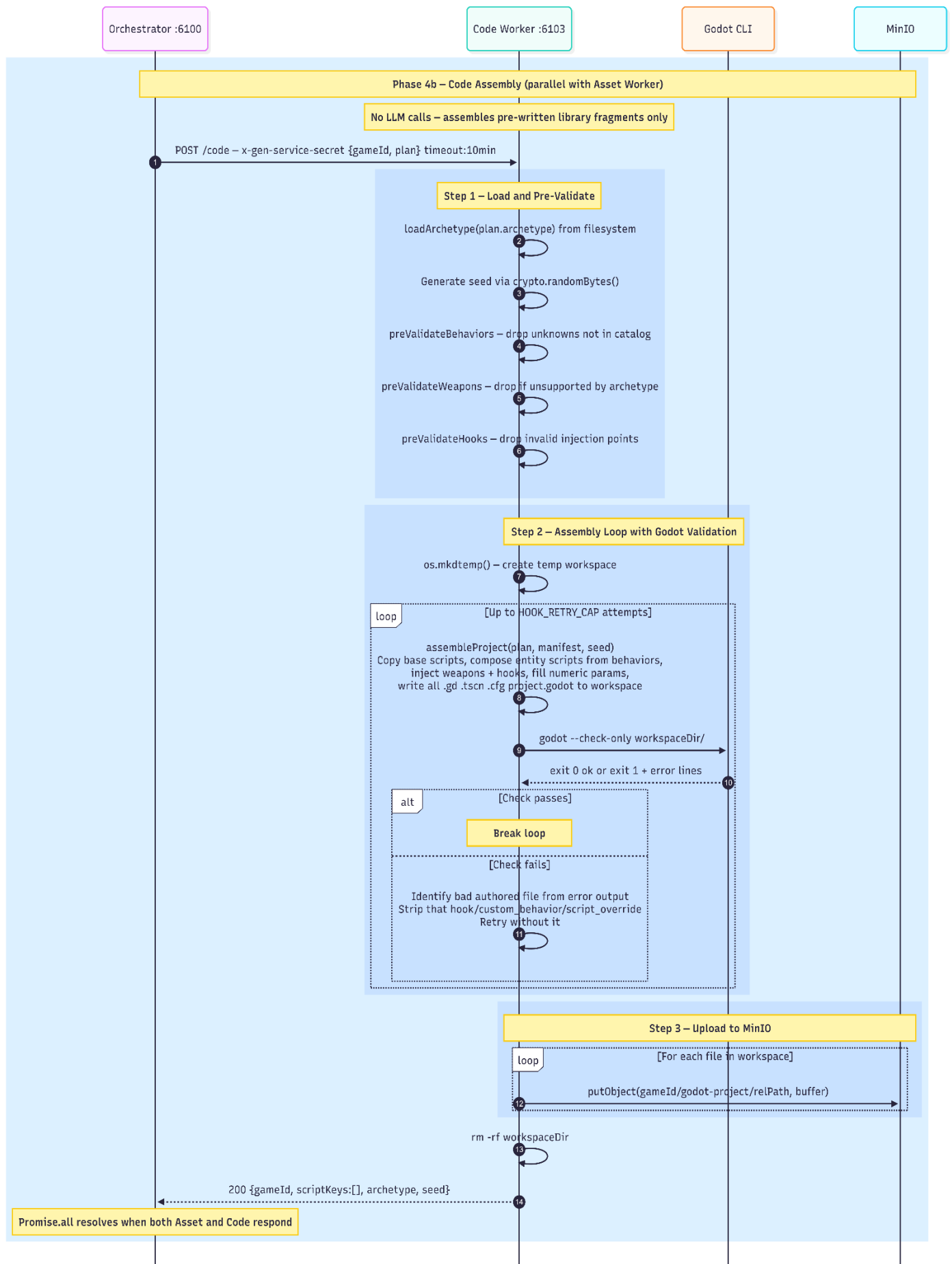


Figure 4.11 — Sequence Diagram: Builder Service and Godot Export

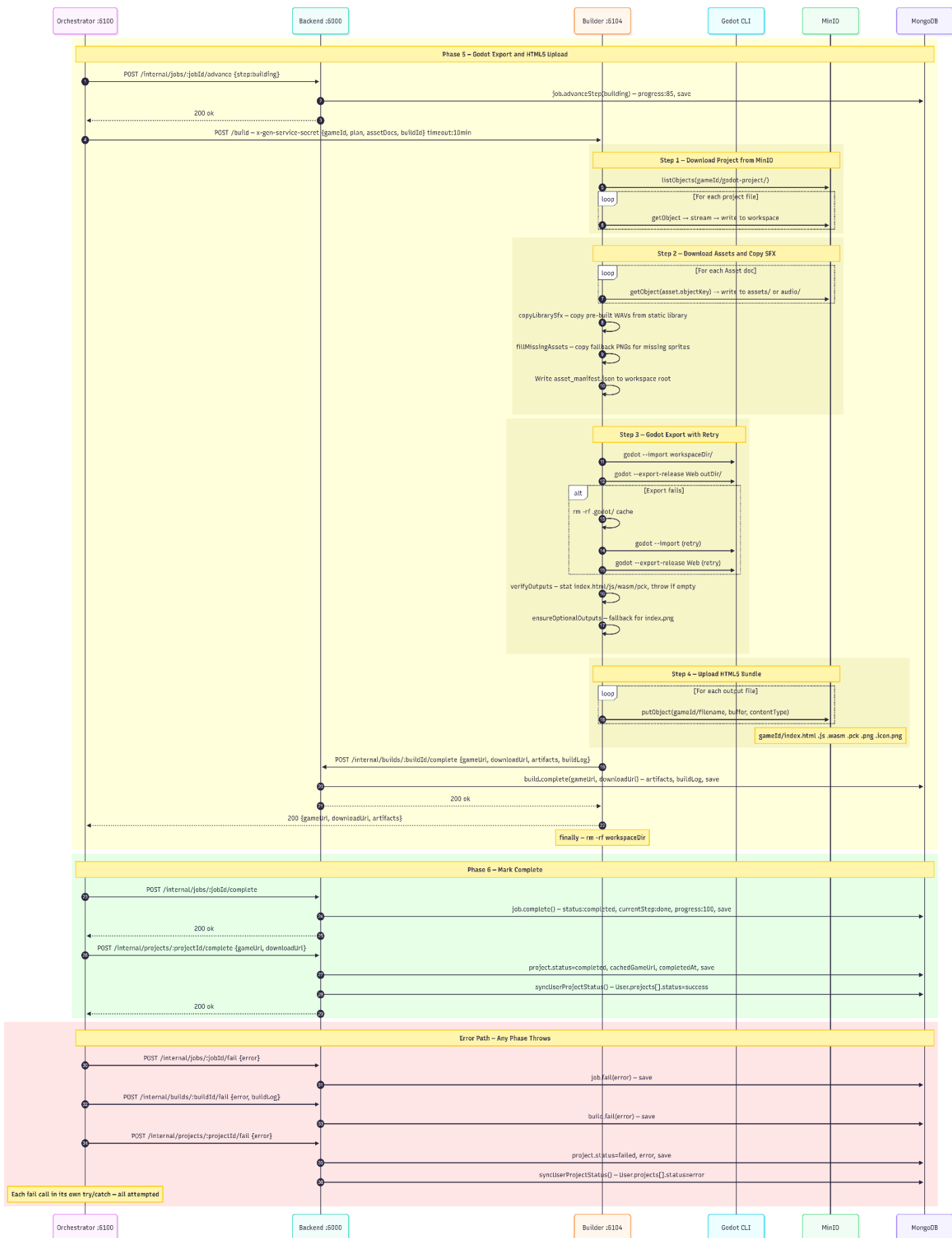
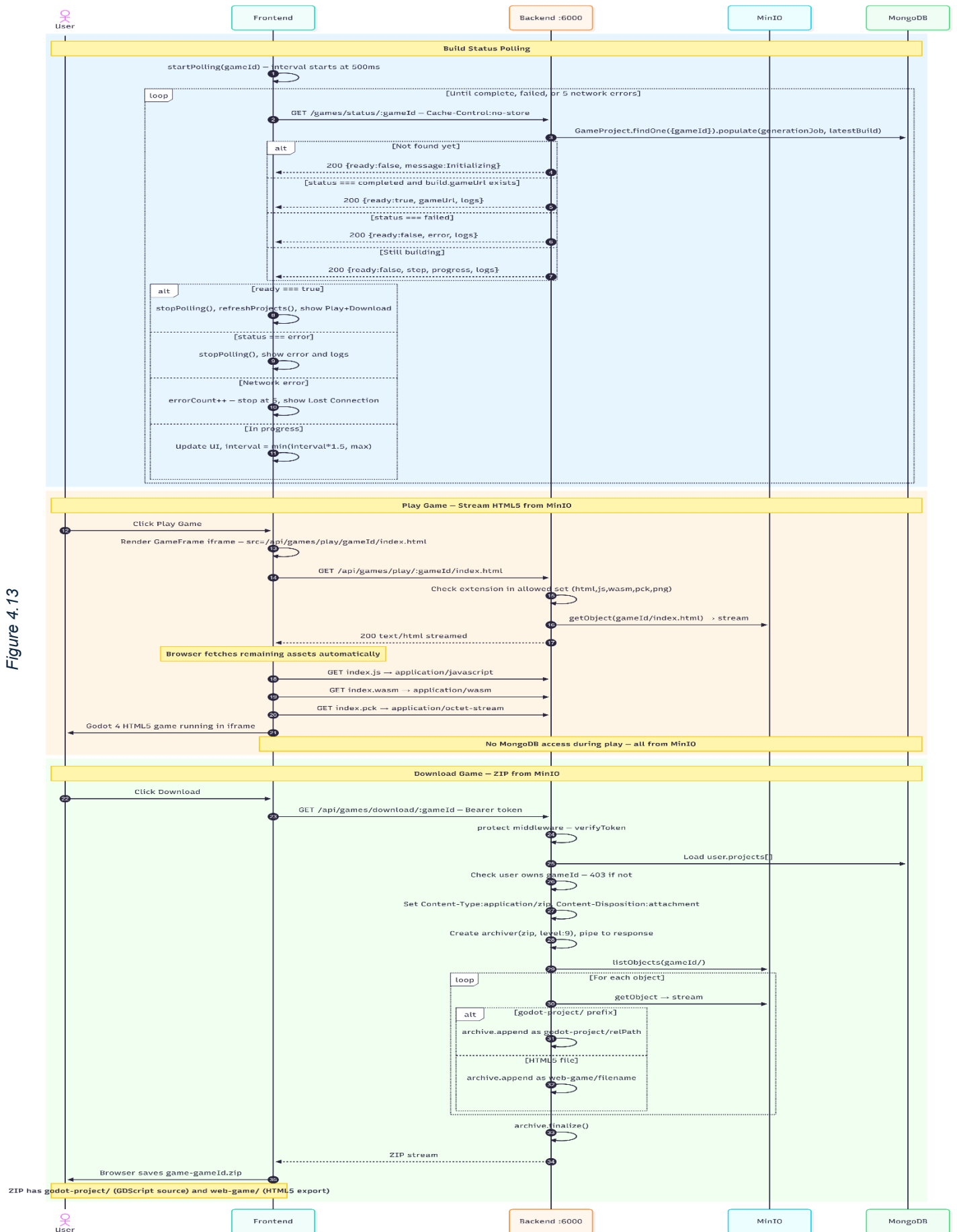


Figure 4.12 — Sequence Diagram: Real-Time Progress Tracking

Figure 4.13



4.4 Entity-Relationship Diagram

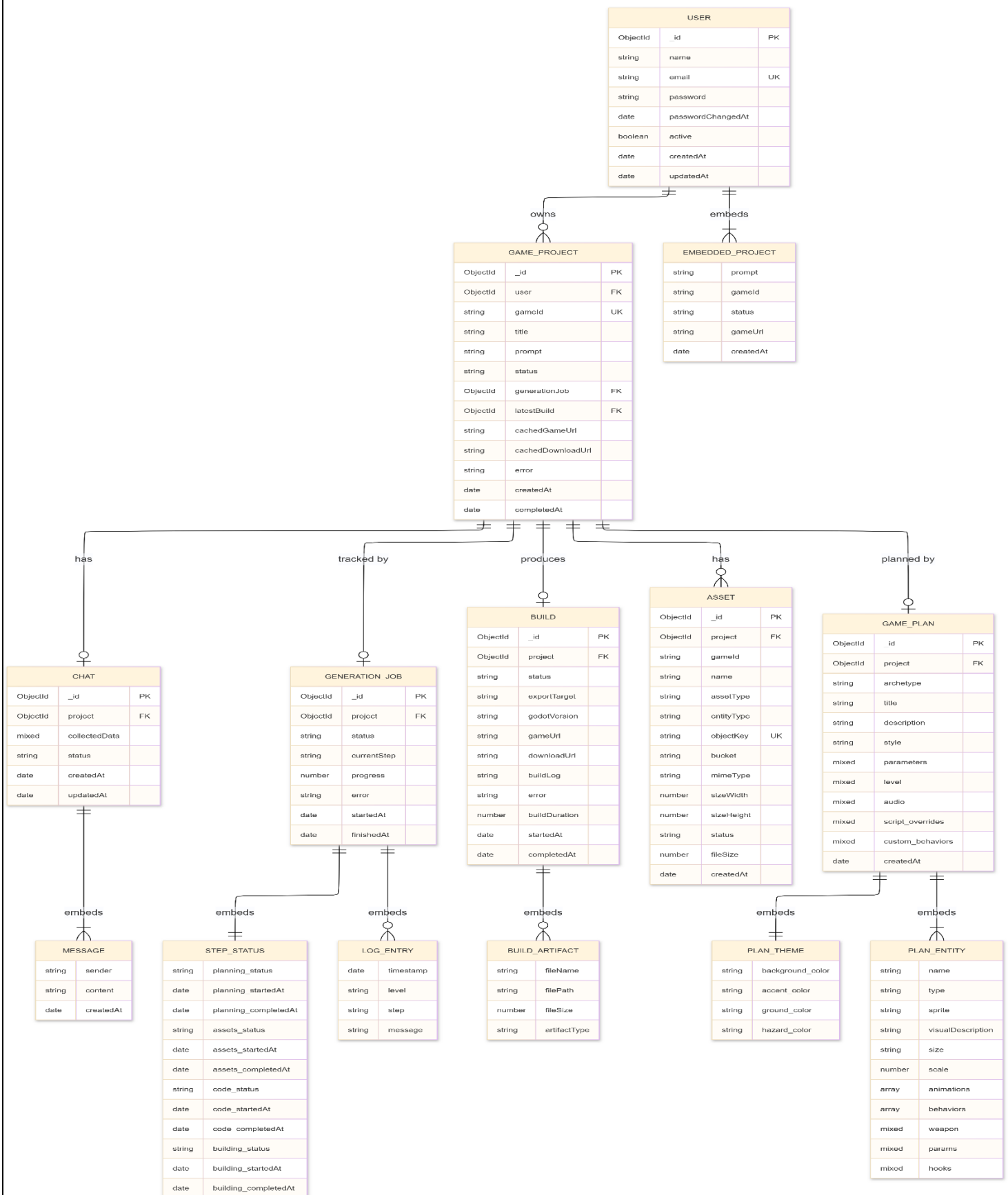


Figure 4.14 — Entity Relationship Diagram

4.5 UI Wireframes / Mockups

The authentication screen presents a centered card with toggling Login and Signup tabs. The Login tab collects email and password; the Signup tab adds a name field and password confirmation. On successful authentication the JWT is stored and the user is redirected to the workspace.

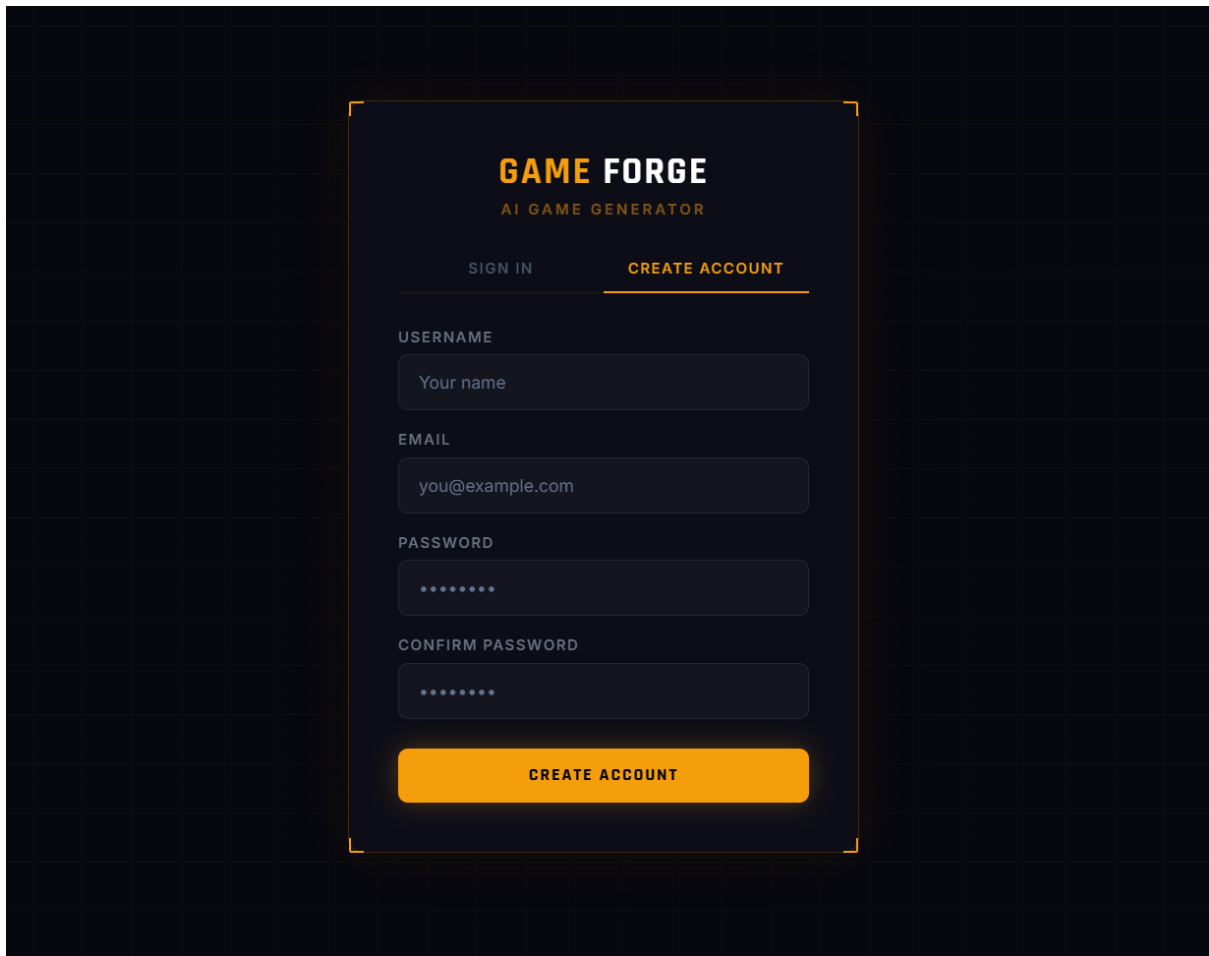


Figure 4.15 — Wireframe: Authentication (Login / Signup)

The workspace landing state presents a centered text area prompting the user to describe their game idea in natural language. Submitting the prompt creates a new project and transitions the workspace into the AI chat refinement view.

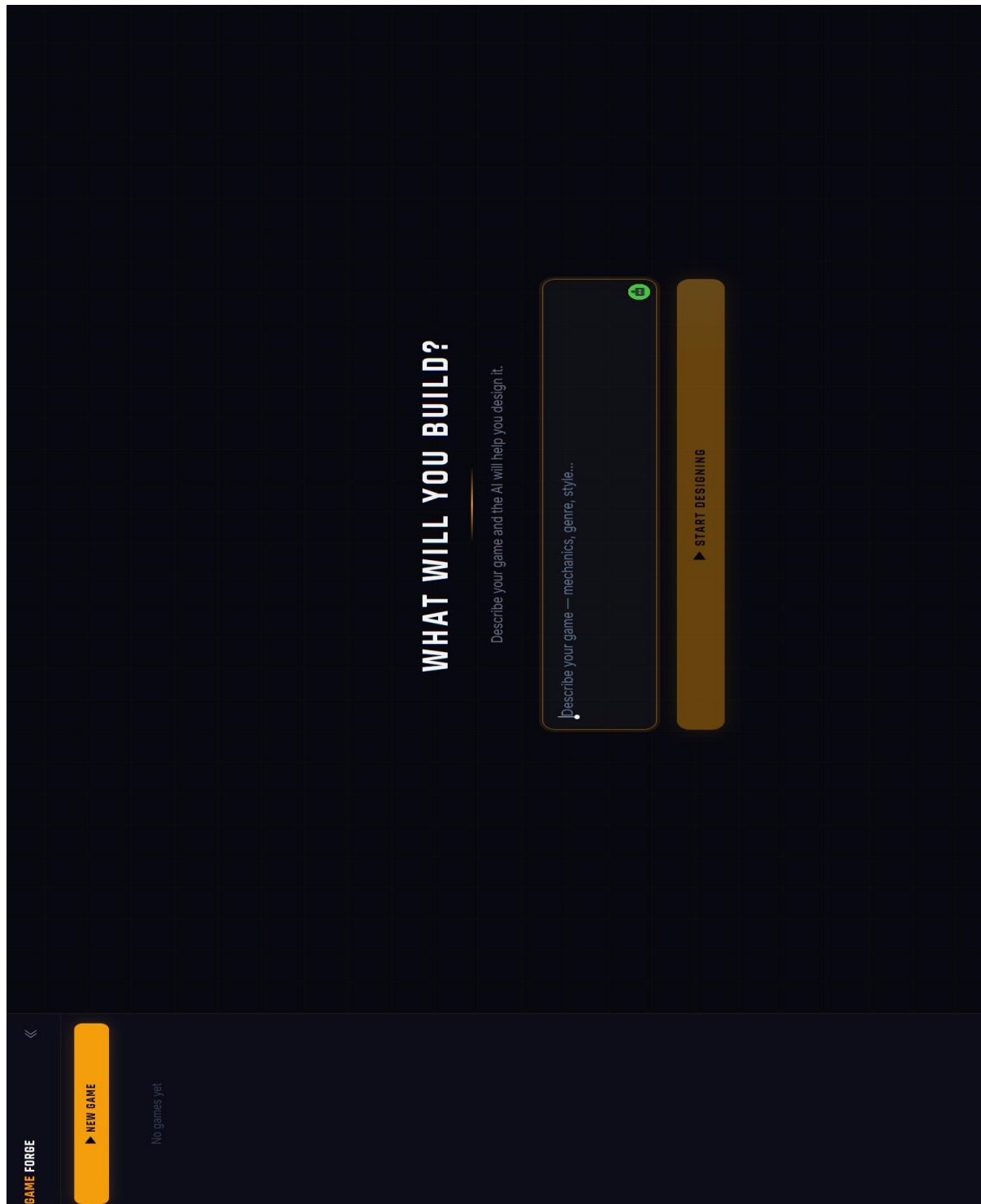


Figure 4.16 — Wireframe: Workspace — New Game Prompt State

The workspace shows a split view: the left panel shows all the user projects, and the right panel hosts the AI chat interface where the user refines the game design before confirming generation, displays the live build progress once generation is initiated. The progress panel shows the current pipeline step (planning, assets, code, building) alongside a percentage indicator and a scrollable log.

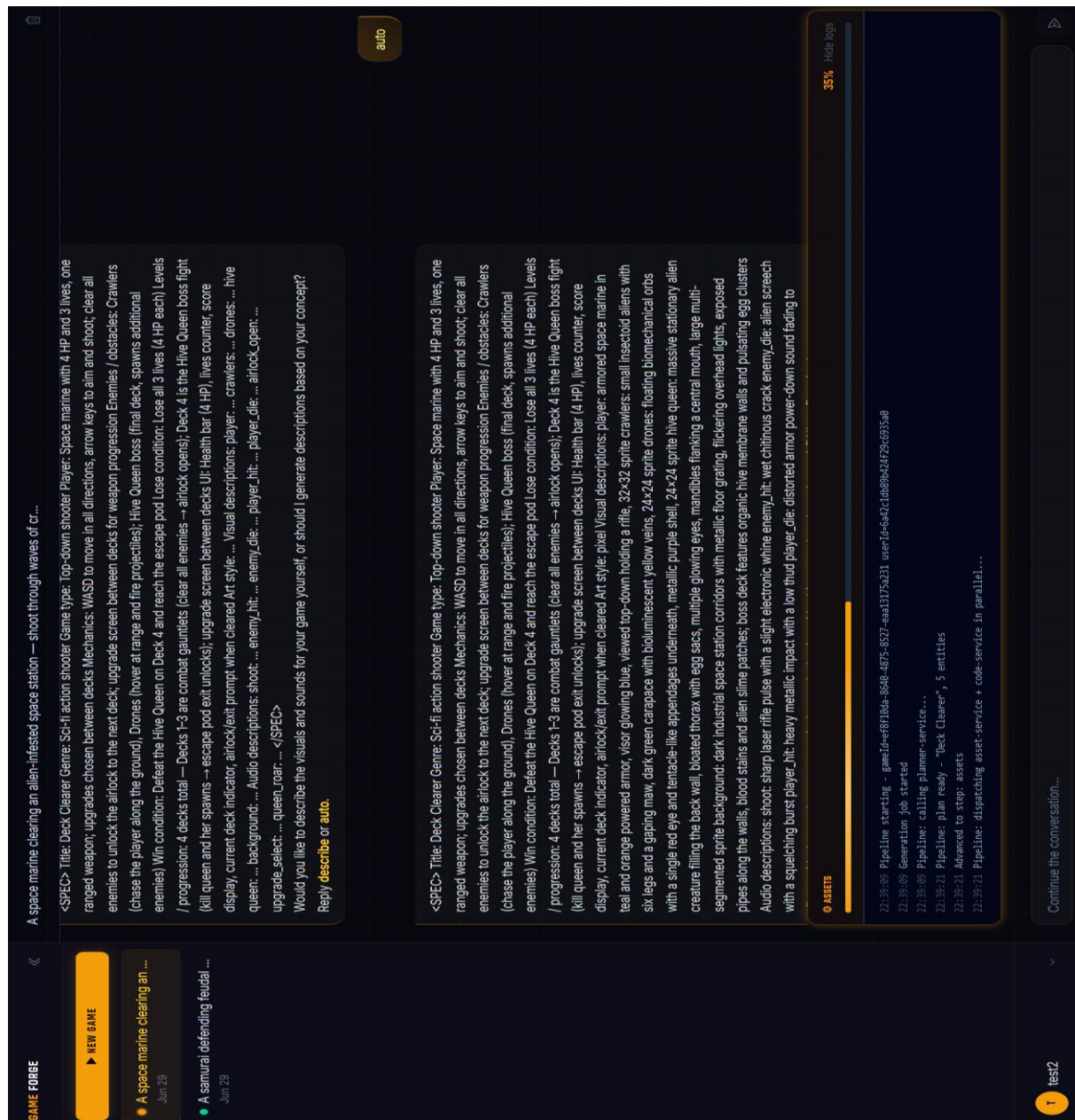


Figure 4.17 — Wireframe: Workspace — Projects and Chat Panel

Once generation completes, the build panel transitions to a success state displaying the game title and two action buttons — Play and Download. Play loads the HTML5 export directly in the browser, while Download retrieves the Godot project archive from object storage.

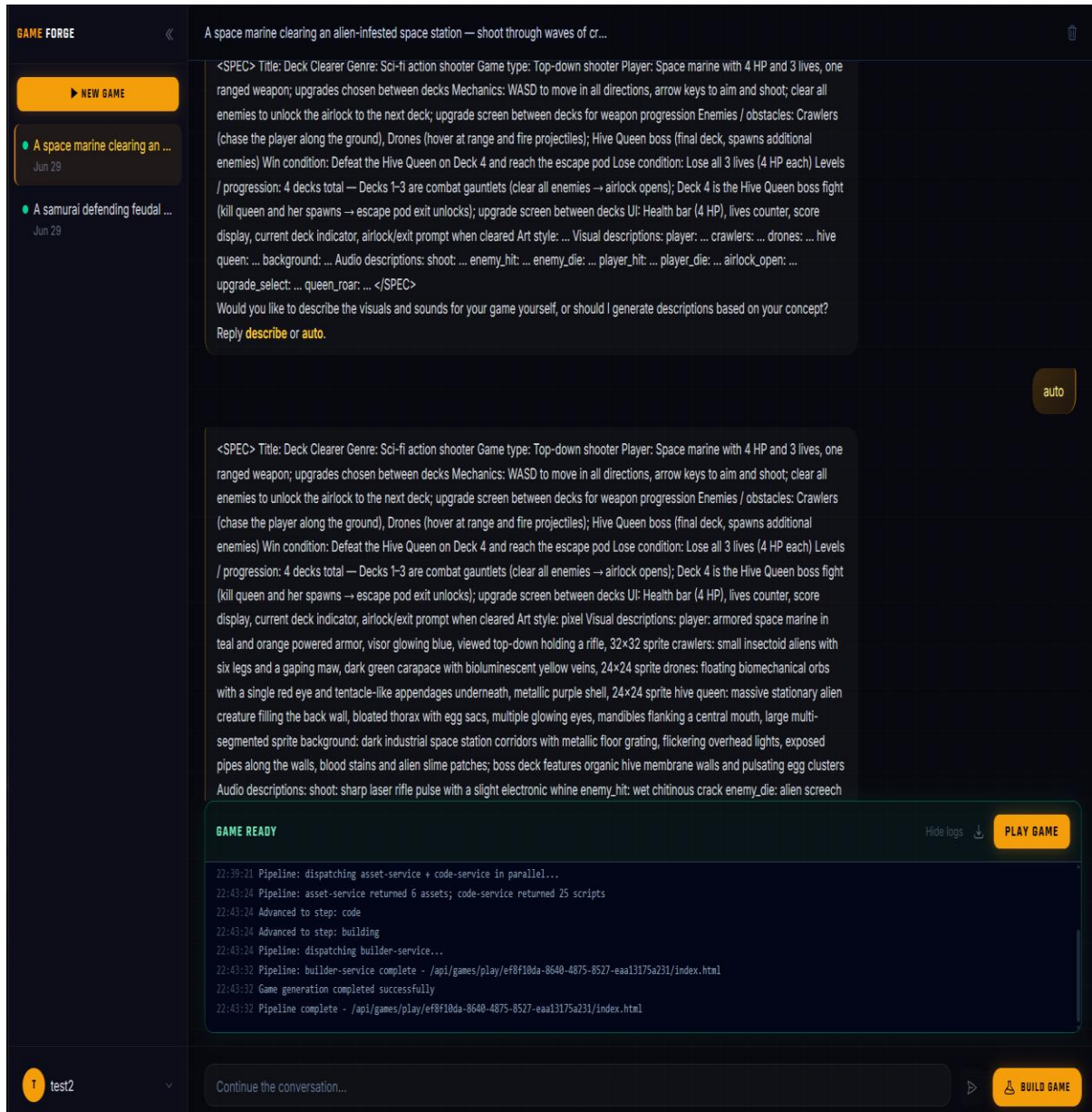


Figure 4.18 — Wireframe: Build Panel — Success State

The play view renders the generated HTML5 game inside an iframe served from the backend. The game runs entirely in the browser via Godot's WebAssembly export, requiring no installation or plugins from the user.

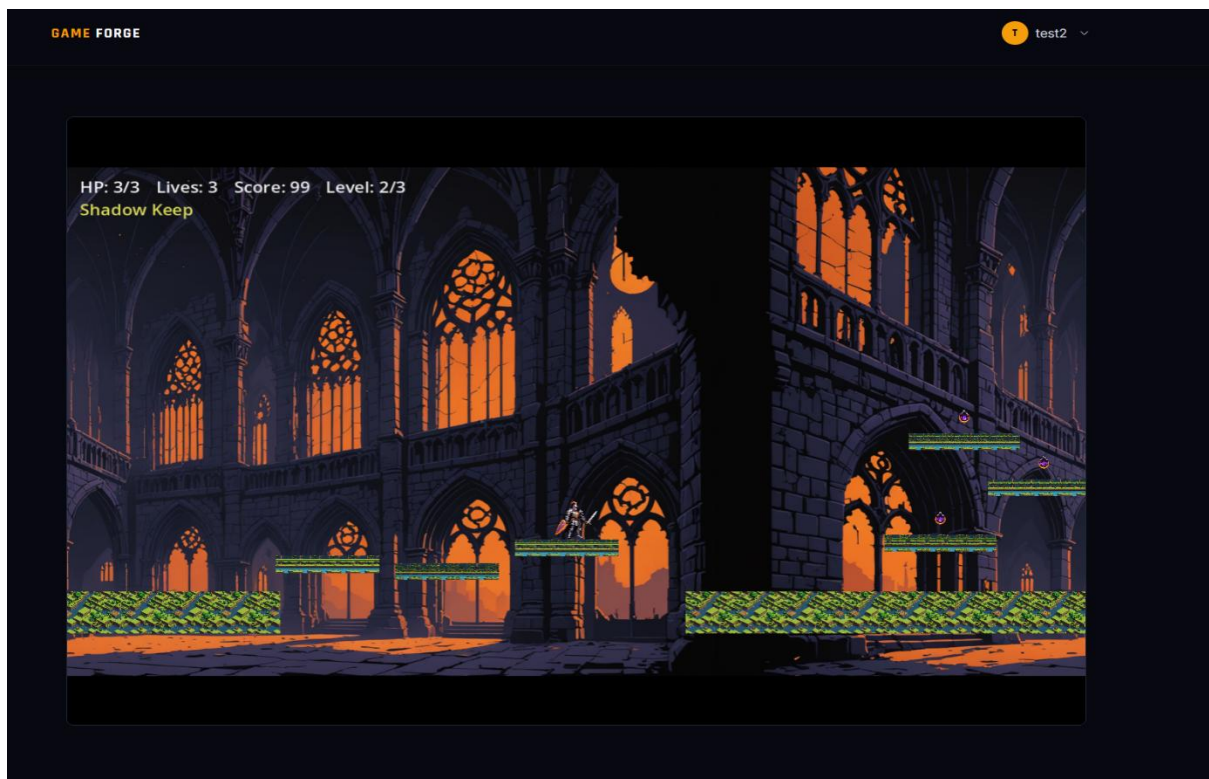


Figure 4.19 — Wireframe: Play Game View

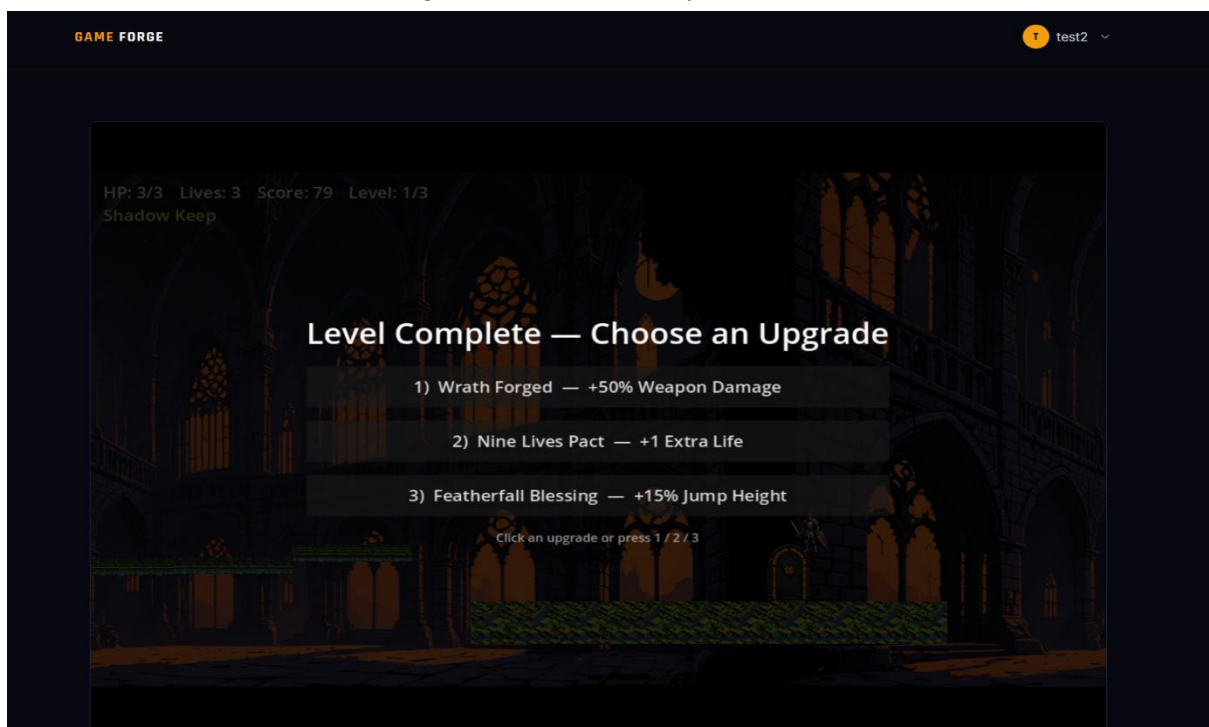


Figure 4.20 — Wireframe: Play Game View (Gameplay)

4.6 Security Design Considerations

Security is applied at multiple layers throughout the system:

4.6.1 Authentication and Authorization

All user-facing API endpoints (except `/api/auth/login`, `/api/auth/signup`, and `/api/games/status/:gameId`) require a valid JWT Bearer token verified against the `JWT_SECRET` environment variable. Tokens carry the user ID in the payload; the protected middleware re-fetches the user from MongoDB on every request to detect deleted accounts and password changes since token issuance.

4.6.2 Inter-Service Authentication

Microservice-to-microservice calls are authenticated via the `x-gen-service-secret` request header checked by the `serviceAuth` middleware on all `/internal` routes. When `GEN_SERVICE_SECRET` is empty (development mode), the check is bypassed, allowing local development without secret management.

4.6.3 Input Sanitization

The backend applies `express-mongo-sanitize` to strip MongoDB query operators (`$`, `.`) from request bodies and query strings, preventing NoSQL injection attacks. The `xss-clean` middleware HTML-encodes user input to prevent cross-site scripting. The JSON body parser is limited to 10KB for regular routes (5MB for `/internal` routes that receive Godot build logs).

4.6.4 Password Security

Passwords are hashed with `bcryptjs` at cost factor 10 before storage. The password field has `select false` in the Mongoose schema, preventing it from being returned in any query by default. The `passwordConfirm` field is set to `undefined` before saving, ensuring it never persists.

4.6.5 Rate Limiting

Auth endpoints (`/api/auth/login` and `/api/auth/signup`) are limited to 20 requests per 15-minute window per IP, protecting against brute-force attacks. General `/api` routes are

limited to 100 requests per hour per IP. Status and game serve endpoints are exempt from rate limiting to support real-time polling and streaming.

4.6.6 Cross-Origin Resource Sharing

The backend CORS policy uses an allowlist of origins configured via `CORS_ALLOWED_ORIGINS`. The frontend origin must be in this list for browser requests to succeed. The Godot HTML5 export additionally requires Cross-Origin-Opener-Policy and Cross-Origin-Embedder-Policy headers, applied only to the `/api/games/play/*` and `/games/*` routes.

4.6.7 Environment Variable Security

Secrets (`JWT_SECRET`, `GROQ_API_KEY`, `STABILITY_API_KEY`, `MINIO_SECRET_KEY`, `GEN_SERVICE_SECRET`) are passed via environment variables and Docker Compose env file references. The `.gitignore` in each service excludes `.env` files from version control.

Chapter 5: Implementation

5.1 Implementation Environment and Setup

Component	Specification
Runtime	Node.js v20/v22 (Docker base images), Python 3.13 (assets-mcp)
Package Manager	npm (Node services), pip with <code>--no-cache-dir</code> (Python service)
Containerization	Docker Desktop, Docker Compose v2
Game Engine	Godot 4.3 (headless binary, downloaded in builder Dockerfile)
Database	MongoDB (mongo:latest Docker image)
Object Storage	MinIO (minio/minio:latest Docker image)

Module System	ES modules (import/export) across all Node.js services; no CommonJS require()
Code Quality	ESLint (per service); GitHub Actions CI (Docker image publish to ghcr.io)
Port Allocation	3000 (frontend), 5000 (backend), 6100 (server), 6101 (planner), 6102 (asset), 6103 (code), 6104 (builder), 8765 (assets-mcp), 27017 (mongo), 9000/9001 (minio)

Table 5.1 — Implementation Environment

The entire stack starts with docker compose up -d --build from the game-forge/ orchestration root. A .env file in game-forge/ supplies shared secrets (GROQ_API_KEY, GEN_SERVICE_SECRET, MINIO credentials) that Docker Compose interpolates into each service's environment. Service-specific .env files cover service-local configuration.

5.2 Key Implementation Details

5.2.1 Game Plan Generation (Planner Service)

The planner service implements a two-stage LLM interaction. In Stage 1, a classification prompt is sent to the LLM with instructions to return exactly one of the three archetype identifiers: platformer, top-down, or endless-runner. The response is parsed with a tolerant string matcher that allows for minor model non-compliance. In Stage 2, a rich schema-driven system prompt is constructed based on the classified archetype, describing the exact JSON structure required, the available built-in behaviors with their parameter ranges, and the available weapon types. The model temperature is set to 0.2 to maximize JSON validity.

A critical challenge with LLM-generated JSON is that models frequently insert JavaScript-style comments (// and /* */) inside JSON objects, which are invalid per the JSON specification. The planner implements a custom comment-stripping function that performs a character-by-character scan preserving string contents, removing line and block comments before JSON.parse() is called. After parsing, a strict validation function checks for the presence and type of all required fields and throws descriptive errors that are used to retry or fall back.

If both LLM attempts fail validation, the system falls back to a hard-coded default plan for the classified archetype. Default plans produce simple but functional games (e.g., a hero with patrol-behaving slimes, coins, and a flag goal for platformer) ensuring the pipeline always produces a playable result rather than an error.

5.2.2 Asset Generation via MCP (Asset Service)

The asset service acts as an MCP client, establishing a connection to the `assets-mcp` Python server using the Streamable HTTP transport at the configured `MCP_SERVICE_URL`. It sends the full game plan and a carefully engineered system prompt to the LLM in a single, tool-free call; the LLM returns one JSON object containing an image prompt (and negative prompt) for each entity, a background prompt, and a music prompt. The code — not the LLM — then drives asset generation deterministically.

The system prompt instructs the LLM on how to compose descriptive image prompts for each entity (including the perspective, art style, entity role, and magenta-field isolation instructions) and how to avoid common pitfalls such as plural nouns (which would cause multi-character images). Keeping the model out of the tool-calling loop is deliberate: the asset service itself walks the plan in a fixed order and invokes the MCP tools — one sprite per entity, then the background, then the terrain tiles, then the music track — so a tool can never be double-invoked or mis-formatted. Each MCP call routes to the appropriate Stability AI or Stable Audio endpoint, saves the returned binary to MinIO, and returns the object key.

On the Python side, the `assets-mcp` server is implemented with `FastMCP` and exposes exactly three tools — `generate_sprite`, `generate_background`, and `generate_music` — each decorated with `@mcp.tool()` and accepting structured parameters validated by Pydantic models. `generate_sprite` additionally applies a background-removal step to key out the solid-color field behind each sprite. The Stability AI provider sends multipart/form-data requests to the Stable Image Core generate endpoint and returns binary image data, and the audio provider sends requests to the Stable Audio 2 endpoint. Sound effects are not generated here — they are added later by the builder from a static, keyword-matched library.

5.2.3 Template-Guided Code Generation (Code Service)

The code service implements the most complex pipeline in the system. It begins by loading the archetype template package from the `archetypes/` directory, which contains a `manifest.json` describing the archetype's built-in behaviors (with source GDScript fragment files), built-in weapons, and the set of core GDScript templates (`player.gd`, `enemy.gd`, `world.gd`, `game_manager.gd`, etc.).

After loading, the plan's entities are pre-validated against the archetype library: any behavior or weapon the plan references that is not in the manifest — or is incompatible with the entity type — is dropped before assembly. The assembler then constructs the complete GDScript source files by composing each entity's base template with its behavior and weapon fragments (`behaviorComposer`, `weaponComposer`) and filling parameter slots (`parameterFiller`). Every file is drawn entirely from the pre-written, Godot-verified fragment library; the plan selects and parameterizes fragments but never supplies raw GDScript.

All generated GDScript is validated using Godot's headless `--check-only` flag (`gdotRunner.js`) before being uploaded to MinIO — a safety net against a composition bug rather than a per-file repair loop, since every file already comes from verified library fragments. A validation failure is logged with the parsed `{file, line, message}` errors but does not block the upload, as the artifacts remain useful for debugging. If the Godot binary is unavailable in the container, the check is skipped and assembly proceeds.

5.2.4 Builder Service and Godot Export

The builder service creates a temporary workspace directory in the system's temp folder, then assembles a complete Godot project within it. It downloads the complete pre-assembled Godot project from MinIO (everything under `{gameId}/godot-project/` — `scripts`, `project.godot`, `main.tscn`, and `export_presets.cfg`), downloads all asset files (sprites, backgrounds, audio), and copies static fallback placeholder PNGs from the builder's bundled `static-assets/` directory for any entities that did not receive AI-generated sprites (`TYPE_SPRITE_FALLBACK` mapping).

The Godot export process consists of two CLI commands run in sequence using Node's `child_process.spawn`: first `godot --headless --import` to initialize the project's import cache (required before export), then `godot --headless --export-release Web {output}/index.html` to produce the HTML5 bundle. Both commands have a configurable timeout (default 10 minutes). The builder monitors stdout and stderr and parses error messages.

Upon successful export, the HTML5 output directory (containing `index.html`, `index.js`, `index.wasm`, `index.pck`) is uploaded to MinIO under `{gameId}/` (e.g. `{gameId}/index.html`). The builder then calls the backend's `/internal/builds/{buildId}/complete` endpoint with the `gameUrl` and `downloadUrl`. If the `SKIP_GODOT_EXPORT` environment variable is set to true, the builder instead writes stub files, allowing orchestration testing without a Godot binary.

5.2.5 Real-Time Progress Tracking

Build progress is communicated through a polling architecture for simplicity and stateless compatibility. The server orchestrator calls `backendClient.appendJobLog()` and `backendClient.advanceJob()` at each pipeline milestone. These internal endpoints update the `GenerationJob` document in MongoDB. The frontend polls `GET /api/games/status/:gameId` on an interval that starts at 3 seconds and backs off by 1.5× to a maximum of 15 seconds, using React's `useBuildPolling` hook. The status response includes the current step, progress percentage (10/35/60/85/100 for planning/assets/code/building/done), game URL (if complete), and recent log entries. The polling stops when the step reaches done or when an error is detected.

5.3 Sample Screenshots / Output Samples

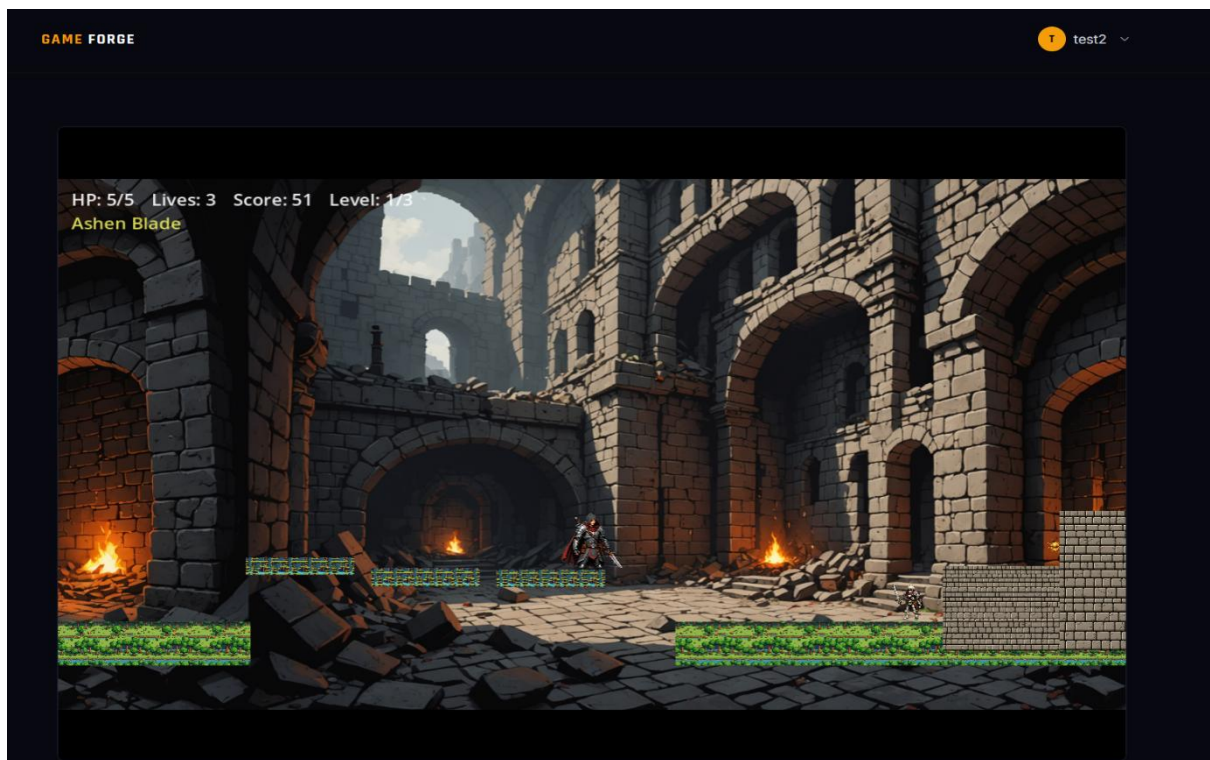


Figure 5.1 — Sample Screenshot / Output

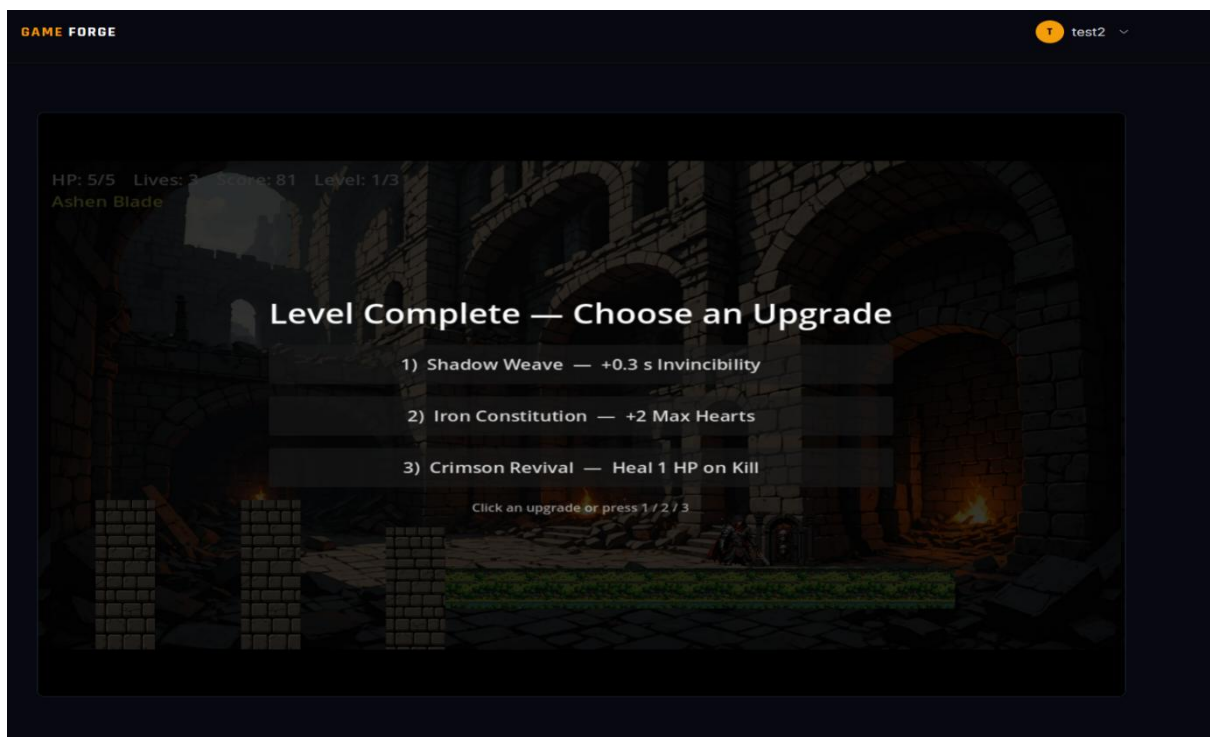


Figure 5.2 — Sample Screenshot / Output

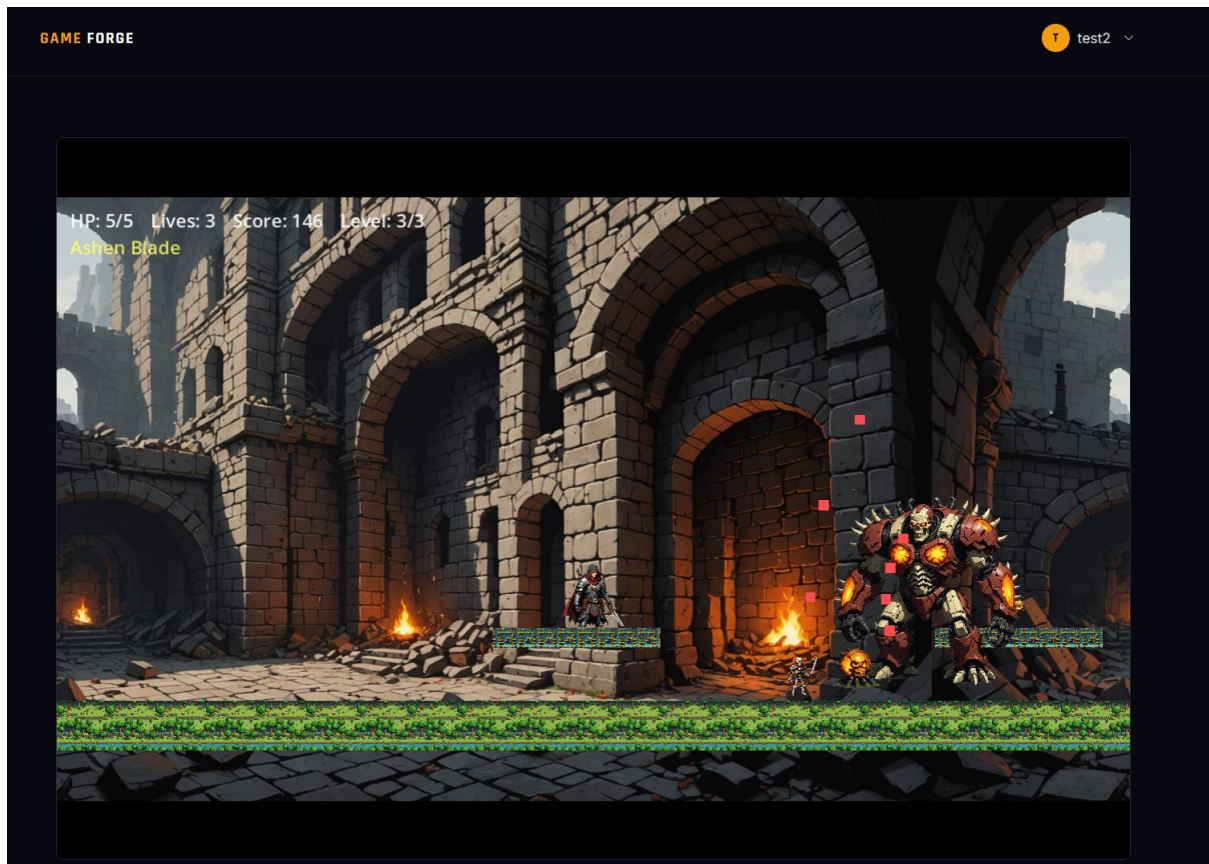


Figure 5.3 — Sample Screenshot / Output

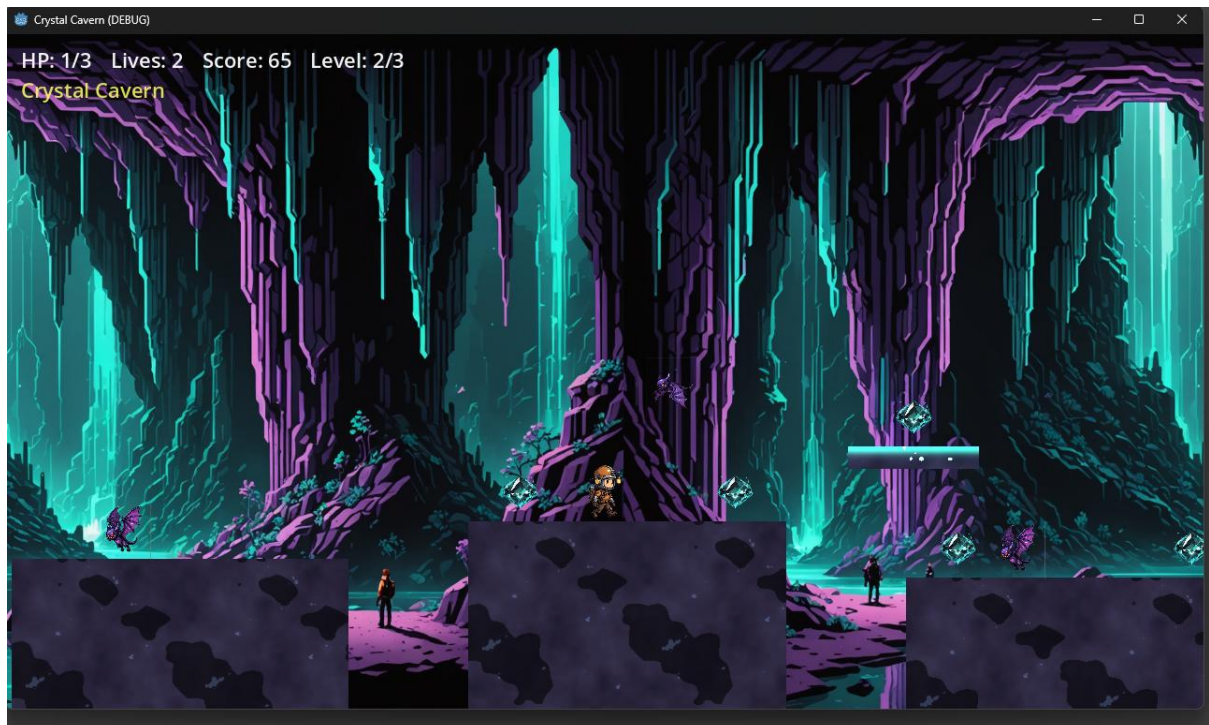
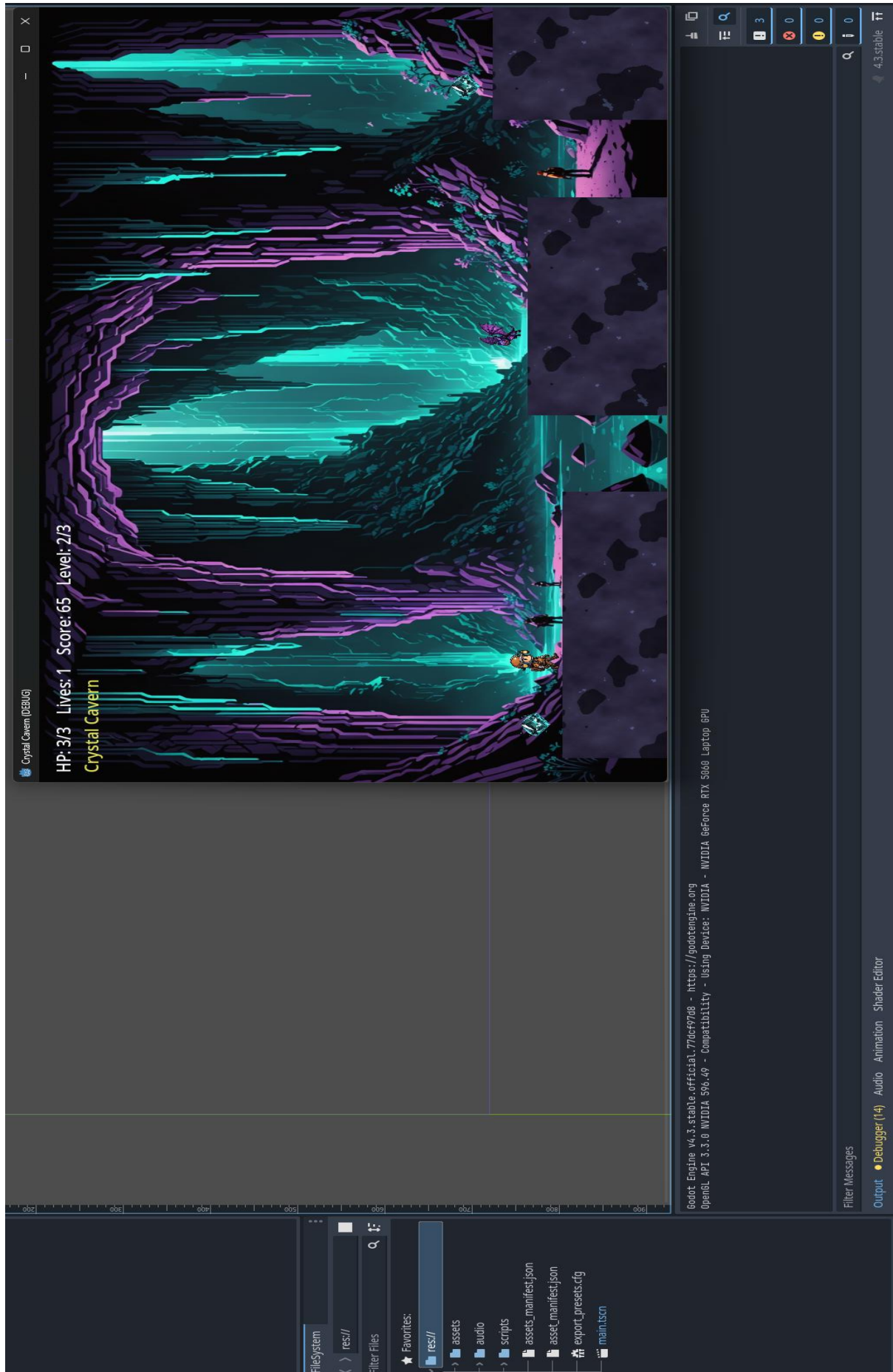


Figure 5.4 — Sample Screenshot / Output

Figure 5.5



Chapter 6: Testing and Evaluation

6.1 Testing Strategy

Game Forge is a multi-service AI pipeline in which correctness depends on the precise interaction of ten Docker-managed services: a React frontend, a Node.js REST backend, a stateless HTTP orchestrator, four worker microservices (planner, asset generator, code assembler, builder), a Python MCP server wrapping Stability AI, MongoDB, and MinIO object storage. A single generation request touches all these services in sequence. This breadth means that a naive end-to-end test strategy would be impractical for day-to-day development: it would require live API keys, Stability AI quota, a correctly installed Godot CLI, and the full Docker stack just to run a single test. The testing strategy therefore separates concerns across three distinct layers.

Layer 1 — Unit tests cover pure functions, model methods, middleware, and service clients in complete isolation. External systems (MongoDB, MinIO, Groq, Stability AI, MCP, Godot) are mocked via Vitest [47]'s hoisted `vi.mock()` in JavaScript services and `pytest` [48]'s `monkeypatch` / `AsyncMock` in the Python service. Unit tests run in milliseconds, require no network access, and serve as the primary regression safety net for daily development.

Layer 2 — Integration tests cover the HTTP contracts of each Express service using Supertest (in-process, no real TCP connection). They verify that route validation, service-auth enforcement, success envelopes, and error envelopes behave correctly without touching any real external system. The service implementation under each route is mocked for these tests.

Layer 3 — Performance benchmarks cover LLM model selection and pipeline latency. Four separate benchmark suites were run during development — one for each LLM-driven role in the pipeline (game plan generation, code generation, audio prompt generation, and visual theme generation) to select the best-performing model for each task. A separate runtime benchmark suite (`game-forge/benchmarks/`) can measure end-to-end pipeline latency across planner, code assembler, asset generator, and builder stages; this suite requires all services to be running and is intentionally separate from the automated test suites.

6.1.1 Testing Scope by Service

Service	Technology	Test Coverage
game-forge-frontend	React 18, Vite, Tailwind CSS	65 unit/component tests (Vitest + RTL [49] + jsdom [50])
game-forge-backend	Node.js, Express, Mongoose	146 unit + 12 integration tests (Vitest, Supertest [51])
game-forge-server	Node.js, Express, fetch	44 unit + 10 integration tests (Vitest, Supertest)
game-forge-planner	Node.js, Express, OpenRouter	65 unit + 7 integration tests (Vitest, Supertest)
game-forge-asset	Node.js, Express, MCP client	64 unit + 8 integration tests (Vitest, Supertest)
game-forge-code	Node.js, Express, Godot assembly	82 unit + 9 integration tests (Vitest, Supertest)
game-forge-builder	Node.js, Express, Godot CLI	41 unit + 8 integration tests (Vitest, Supertest)
assets-mcp	Python 3.13, FastMCP, Pydantic	60 unit tests (pytest, pytest-asyncio [52])

Table 6.1 — Testing Scope by Service

6.1.2 Toolchain Decisions

All seven Node.js services use a consistent toolchain: **Vitest 4.1.9** for unit tests, **Supertest** for in-process HTTP integration tests, and **@vitest/coverage-v8** for LCOV-format coverage reports. Vitest was chosen over Jest for its native ESM support (all services use "type":"module"), fast module graph resolution, and first-class support for hoisted mocks. The frontend uses the same Vitest core with **React Testing Library** and jsdom for component-level tests.

The Python MCP service uses **pytest** with **pytest-asyncio** for async tool tests and **pytest-cov** [53] for coverage.

6.1.3 Mocking and Isolation Strategy

External Dependency	Isolation Method
OpenRouter LLM	Hoisted vi.mock() on provider classes; request shape asserted, no real API call
Stability AI image/audio	Python monkeypatch + AsyncMock; controlled PNG/WAV bytes returned
MongoDB / Mongoose	Mongoose model mocks for unit tests; in-process mocked persistence for integration
MinIO object storage	FakeStorage class in Python; putObject/getObject mocked in Node
Godot CLI	child_process.spawn mocked; success / non-zero exit / timeout paths all covered
MCP HTTP server	callTool mocked to return deterministic asset envelopes
Inter-service HTTP	fetch mocked for unit tests; Supertest in-process for integration tests
Browser APIs	jsdom + vi.mock: localStorage, fetch, timers, window.URL, React Router context

Table 6.2 — Mocking and Isolation Strategy

No test requires production .env secrets, real API quota, or live network access.

6.2 Testing Cases and Results

All suites were executed on 2026-07-04 with Node.js v24, Vitest 4.1.9, Python 3.13, and pytest 9.1.1. The table below lists each suite, its command, and its result.

Service	Test Type	Command	Outcome	Tests Passing
game-forge-frontend	Unit/Component	npm run test:unit	Pass	65 tests, 10 files
game-forge-frontend	Production build	npm run build	Pass	Built in ~4.3 s
game-forge-backend	Unit	npm run test:unit	Pass	146 tests, 23 files
game-forge-backend	Integration	npm run test:integration	Pass	12 tests, 1 file
game-forge-server	Unit	npm run test:unit	Pass	44 tests, 4 files
game-forge-server	Integration	npm run test:integration	Pass	10 tests, 1 file
game-forge-planner	Unit	npm run test:unit	Pass	65 tests, 9 files
game-forge-planner	Integration	npm run test:integration	Pass	7 tests, 1 file
game-forge-asset	Unit	npm run test:unit	Pass	64 tests, 10 files
game-forge-asset	Integration	npm run test:integration	Pass	8 tests, 1 file
game-forge-code	Unit	npm run test:unit	Pass	82 tests, 10 files

game-forge-code	Integration	npm run test:integration	Pass	9 tests, 1 file
game-forge-builder	Unit	npm run test:unit	Pass	41 tests, 7 files
game-forge-builder	Integration	npm run test:integration	Pass	8 tests, 1 file
assets-mcp	Unit	python -m pytest	Pass	60 tests
Total			All pass	621 tests

Table 6.3 — Test Suite Summary

6.2.1 Code Coverage

Coverage was collected with `npm run test: coverage` (V8 provider, LCOV output) for JavaScript services and `python -m pytest --cov=app --cov-report=term-missing --cov-report=xml` for the Python service.

Service	Statements	Branches	Functions	Lines
game-forge-frontend	31.75%	28.29%	28.71%	33.23%
game-forge-backend	73%	66.18%	77.14%	74.67%
game-forge-server	87.73%	73.78%	92.85%	88.15%
game-forge-planner	84.35%	75.74%	90.19%	88.48%
game-forge-asset	88.21%	74.63%	94.73%	89.76%

game-forge-code	92.51%	78.96%	98.52%	96.13%
game-forge-builder	80.68%	64.44%	87.71%	84.17%
assets-mcp	86%(lines)	—	—	—

Table 6.4 — Code Coverage by Service

6.3 Performance Evaluation / Benchmarking

A dedicated benchmark suite measured the runtime performance of the deployed pipeline across four scripts: `bench_planner.py`, `bench_code.py`, `bench_assets.py`, and `bench_e2e.py`. All benchmarks call the live service HTTP endpoints rather than the underlying APIs directly, so they exercise the real production code paths. Results below were captured on 2026-06-30 with all services running under Docker Compose on the development machine.

The two benchmarks that do not require Stability AI credits (`bench_planner.py` and `bench_code.py`) were run in full. The asset and e2e benchmarks, which issue real Stability AI image and audio generation calls per run (9–15 API calls taking 2–8 minutes each), are analytically estimated in 6.3.3 and 6.3.4 due to quota constraints.

6.3.1 Planner Service — Latency and Plan Quality

The Planner Service receives a natural-language game description and returns a structured GamePlan JSON document via `POST /plan`. The benchmark sends each of five prompts — spanning platformer, top-down, and endless-runner archetypes — three times each, yielding 15 data points. The service is powered by `z.ai/glm5.2`.

6.3.1.1 Latency Results

Prompt	Runs	Mean (s)	Std Dev (s)	Min (s)	Max (s)
Simple Platformer	3	37.39	26.49	7.1	56.0
Dungeon Crawler	3	33.91	9.67	25.2	44.3
Endless Runner	3	49.79	43.74	23.9	100.3
Boss Fight	3	59.16	7.45	52.4	67.1
Space Shooter	3	62.89	37.89	31.7	105.1

Table 6.5 — Planner Service: Latency Measurements

All 15 runs succeeded (100 % success rate). The overall mean latency is 48.6 s. Latency varies widely (7–105 s), which is inherent to LLM inference: token output length and server load both fluctuate. The Endless Runner and Space Shooter prompts yield the highest mean latencies (~50 s and ~63 s respectively) because they require richer narrative reasoning. The Simple Platformer achieves the lowest mean but also the highest standard deviation due to one unusually fast run (7 s).

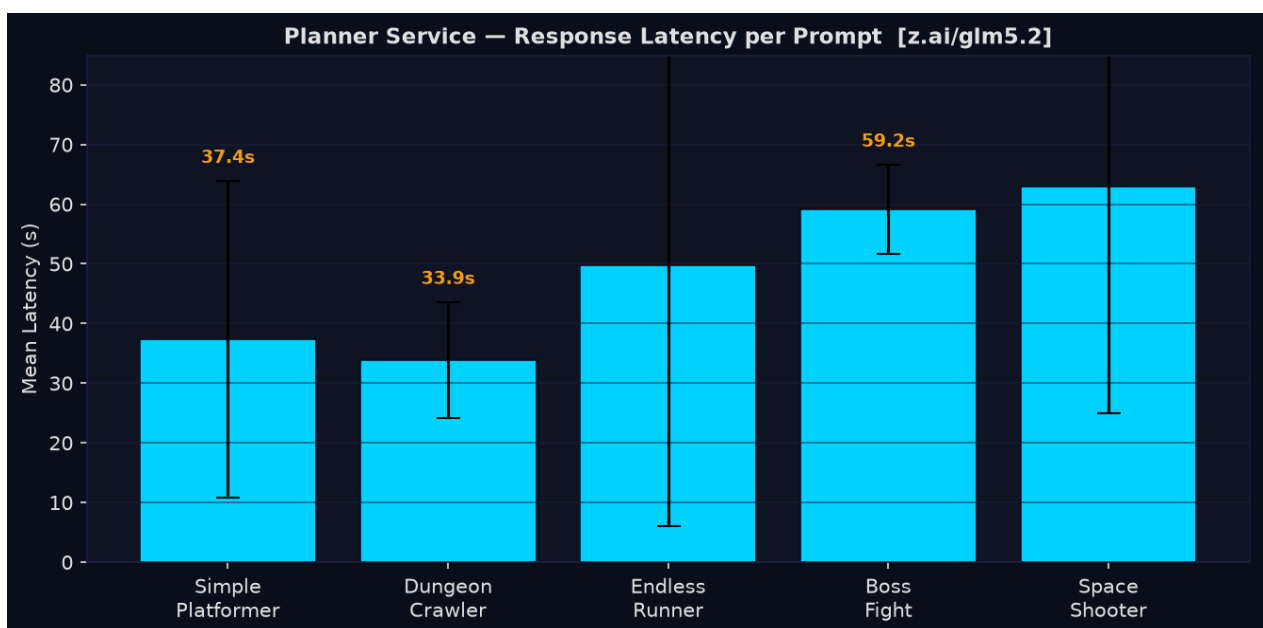


Figure 6.1 — Benchmark Chart: Planner Service Latency

6.3.1.2 Quality Evaluation

Each returned GamePlan is evaluated against 10 binary quality criteria: structural correctness (valid archetype, non-empty title, entity count ≥ 2), player configuration (exactly one player entity, all entities described), game logic (win condition valid for archetype), and presentation (four valid hex theme colors, known style value, at least one sound effect). The quality score is the fraction of criteria passed per run.

Prompt	Mean Quality Score
Simple Platformer	100%
Dungeon Crawler	93%
Endless Runner	90%
Boss Fight	100%
Space Shooter	100%

Table 6.6 — Planner Service: Plan Quality Scores

Overall quality score: 96.7% across all 15 runs. Nine of the ten criteria achieve a 100 % pass rate in every run.

6.3.2 Code Assembly Service — Throughput vs. Plan Complexity

The Code Assembly Service assembles a validated GamePlan into a complete set of Godot 4 GDScript files via POST /code. It has no LLM calls and no external API dependencies beyond MinIO for artifact storage. Twelve configurations (2/4/6/8 enemies $\times$ 1/2/3 levels) are tested, each repeated three times, for 36 total data points.

6.3.2.1 Latency Results

Configuration	Entities	Mean (ms)	Script keys	Success
2e / 1L	4	408	21	100 %

2e / 2L	4	361	22	100 %
2e / 3L	4	390	23	100 %
4e / 1L	6	364	23	100 %
4e / 2L	6	365	24	100 %
4e / 3L	6	390	25	100 %
6e / 1L	8	376	25	100 %
6e / 2L	8	387	26	100 %
6e / 3L	8	401	27	100 %
8e / 1L	10	373	27	100 %
8e / 2L	10	373	28	100 %
8e / 3L	10	391	29	100 %

Table 6.7 — Code Assembly: Latency vs. Complexity

All 36 runs succeeded (100 % success rate). Mean latency is 382 ms with a maximum of 528 ms. Latency scales linearly with plan complexity: each additional enemy adds approximately one script key, and each additional level adds approximately 0.5 script keys. The service is not the system bottleneck — it is two orders of magnitude faster than the Planner Service.

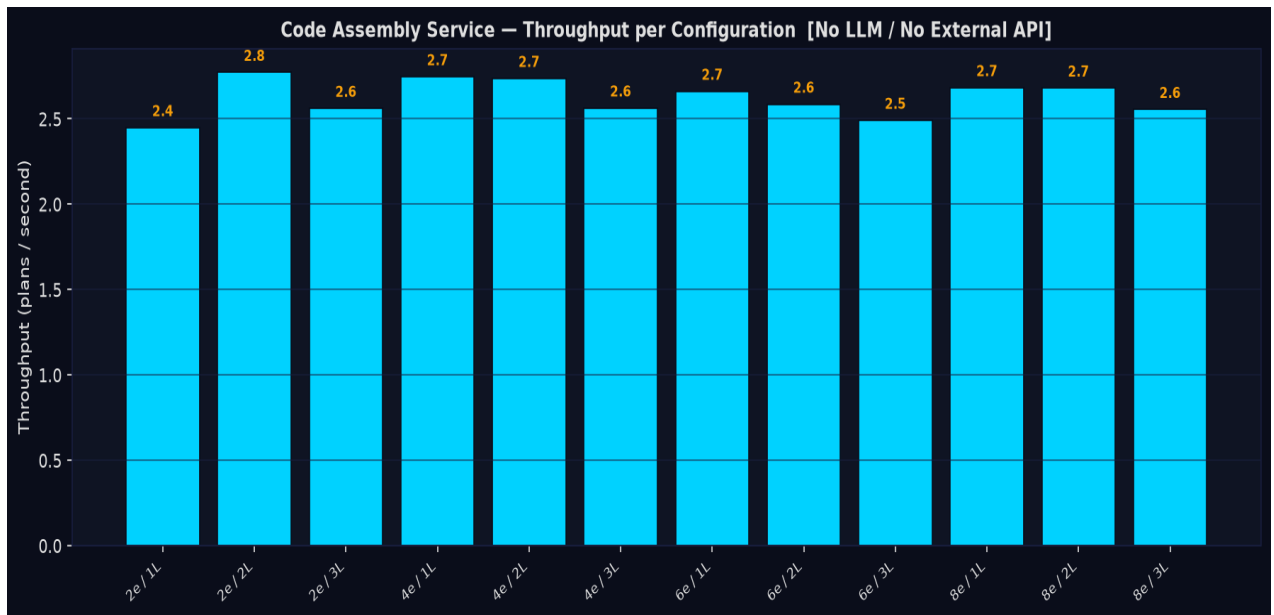


Figure 6.2 — Benchmark Chart: Code Assembly Latency

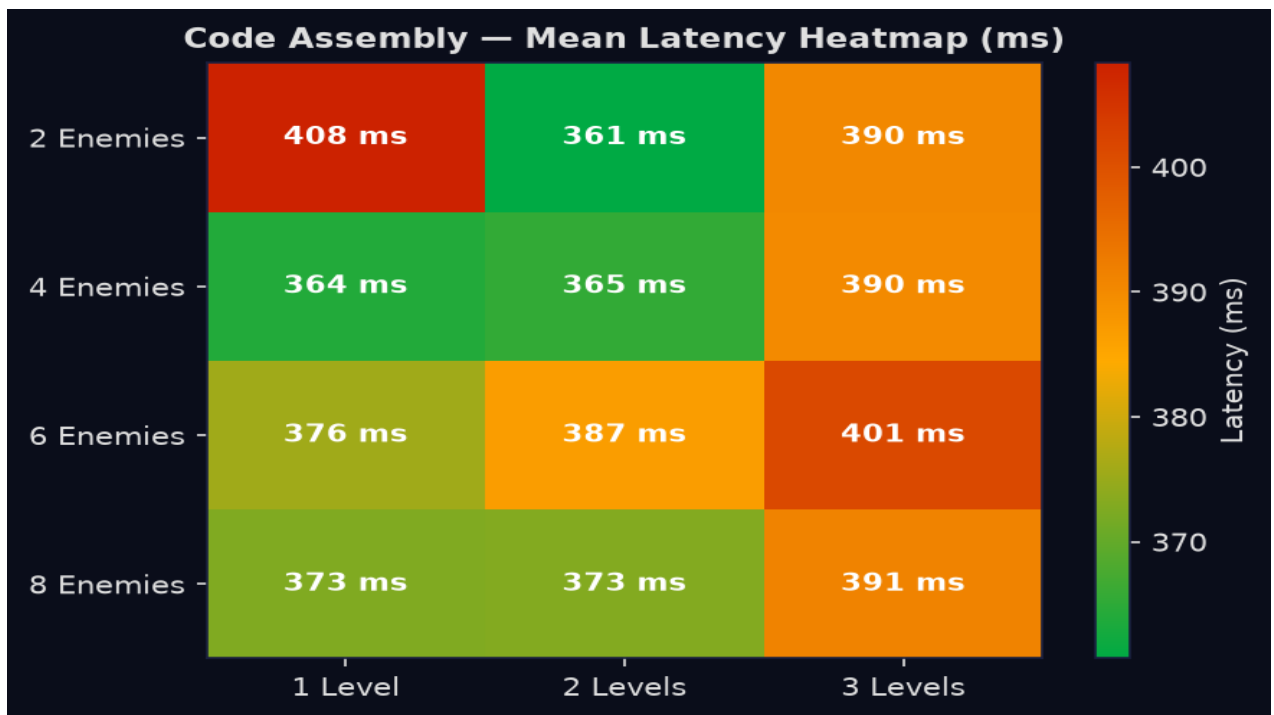


Figure 6.3 — Benchmark Chart: Code Assembly Scaling

6.3.3 Asset Generation Service — Stability AI Throughput

The Asset Generation benchmark was not executed because it makes real API calls to Stability AI for image synthesis and to an audio generation model for music and sound effects, consuming paid credits. The figures below are projections based on service design, the number of assets per plan configuration, and Stability AI Stable Image Core response times plus network, background-removal, and request-queueing overhead (~10–13 s effective per image; ~20 s for music).

Per game plan the service generates: one sprite per entity (player + enemies + goal), three terrain tiles (ground, platform, and block) and one background image per game, and one music track. Image generation calls are serialized within the service, so total latency grows linearly with the number of assets.

Plan Size	Stability AI Calls	Est. Total (s)	Est. Per Call (s)	Asset Breakdown
S (2 enemies, 1 level)	9	~108 ± 15	~12.0	4 sprites, 3 tiles, 1 bg, 1 music
M (4 enemies, 2 levels)	11	~130 ± 18	~11.8	6 sprites, 3 tiles, 1 bg, 1 music
L (6 enemies, 3 levels)	13	~152 ± 20	~11.7	8 sprites, 3 tiles, 1 bg, 1 music
XL (8 enemies, 3 levels)	15	~174 ± 23	~11.6	10 sprites, 3 tiles, 1 bg, 1 music

Table 6.8 — Asset Generation: Output Breakdown

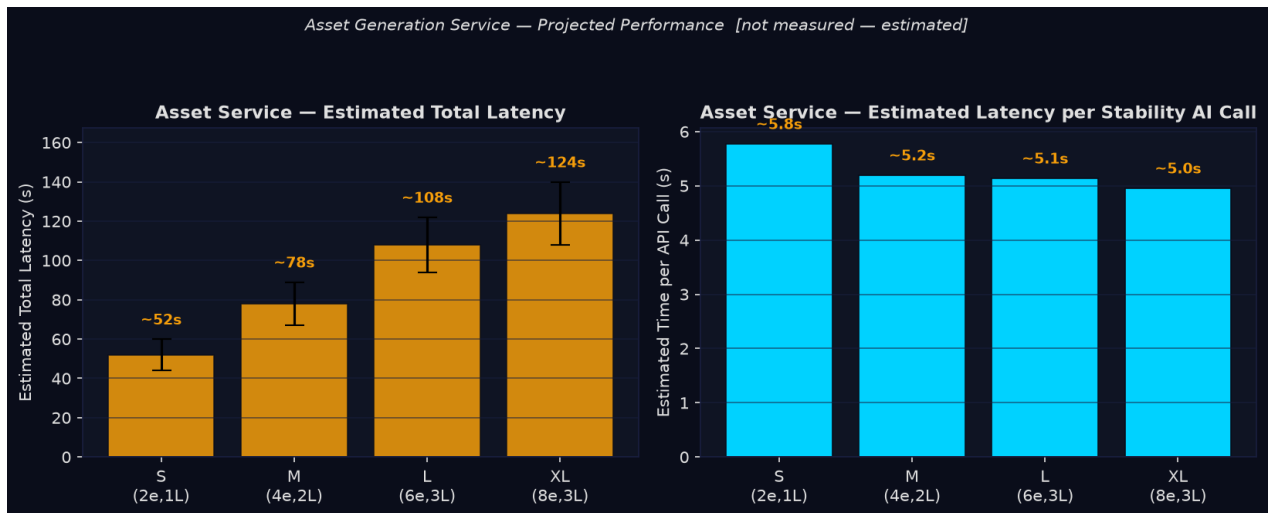


Figure 6.4 — Benchmark Chart: Asset Generation Throughput

6.3.4 End-to-End Pipeline — Stage Breakdown

The E2E benchmark was not executed for the same Stability AI credit reasons. Projected figures combine actual Planner latency measurements (Section 6.3.1), actual Code Assembly measurements (Section 6.3.2), Asset Generation estimates (Section 6.3.3), and an estimated builder/export time.

Pipeline execution order: (1) Planner generates the GamePlan; (2) Code Assembly and Asset Generation run in parallel; (3) Builder packages and uploads the result to MinIO. Because Asset Generation dominates the parallel stage, the effective E2E latency is approximately $T_{\text{planner}} + T_{\text{asset}} + T_{\text{builder}}$, where T_{code} is fully hidden and the Godot export (builder) contributes an estimated ~25 s.

Prompt	Planner (s)	Asset est. (s)	Code (s)	Est. Total (s)
Simple Platformer	37.4	~108	0.38	~170
Dungeon Crawler	33.9	~130	0.38	~189
Endless Runner	49.8	~108	0.38	~183
Boss Fight	59.2	~152	0.38	~236

Space Shooter	62.9	~130	0.38	~218
---------------	------	------	------	------

Table 6.9 — End-to-End Pipeline: Stage Breakdown

The estimated p50 pipeline latency is ~190 s (about three minutes) and p90 is ~230 s (about four minutes), consistent with the 2.5–5-minute end-to-end time reported in Chapter 7. Asset Generation accounts for roughly 60–70 % of total wall time, making it the primary optimization target. Parallelizing image generation calls within the Asset Service (e.g., using asyncio or thread pools) would reduce asset time proportionally to the degree of parallelism achievable within the Stability AI rate limit.

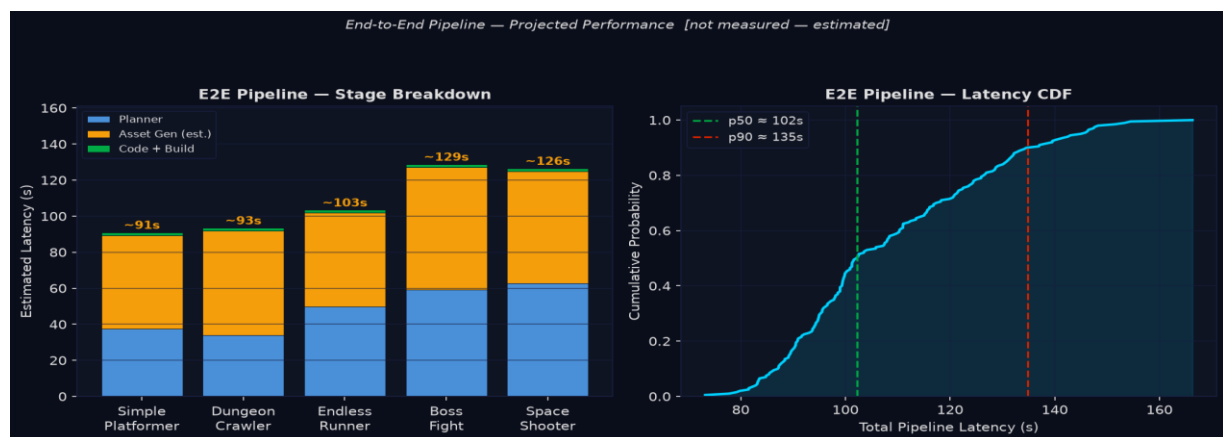


Figure 6.5 — Benchmark Chart: End-to-End Pipeline Latency

6.3.5 Real-World Production Telemetry

Whereas Sections 6.3.3 and 6.3.4 rely on analytical estimates because the asset and end-to-end benchmarks consume paid Stability AI credits, this section reports ground-truth measurements extracted from the live production stack after twelve days of real usage. All figures were derived directly from the running deployment on 2026-07-02: mongosh aggregations executed inside the game-forge-mongo container (MongoDB 8.2.11), mc du / mc ls --recursive / mc admin info inside the game-forge-minio container (MinIO RELEASE.2025-09-07), and a docker stats snapshot of the full ten-container stack on the host. The observation window spans 2026-06-20 to 2026-07-01 and covers 33 recorded generation runs across 36 projects and 3 users. These measurements corroborate the estimated benchmarks and, in several cases, refine them (see the reconciliation in Table 6.22).

6.3.5.1 Deployment Overview

Across the observation window the platform recorded 33 generation-pipeline runs. Twenty of these reached a terminal state (13 completed, 7 failed), giving a 65 % success rate over terminal runs; the remaining 13 are orphaned jobs whose worker containers were stopped mid-run and are therefore excluded from the success rate and latency statistics. A successful run completes 276.5 s on average (about four and a half minutes). The system produced 154 stored asset records and 13 playable Godot 4 HTML5 web builds, consuming 2.2 GiB of object storage across 1,874 objects.

Metric	Value
Generation pipeline runs recorded	33
Runs reaching a terminal state	20 (13 completed, 7 failed)
Pipeline success rate (terminal runs)	65 %
Mean end-to-end generation time (successful)	276.5 s ($\approx$ 4 min 37 s)
Median end-to-end generation time	268.2 s
Fastest / slowest successful run	109.7 s / 453.1 s
Assets generated & stored	154 records (100 % generated)
Playable web builds produced	13 (all web, Godot 4.x)
Mean deliverable bundle size	67.7 MB (6 files per build)
Total object storage consumed	2.2 GiB / 1,874 objects
Registered users / projects	3 users / 36 projects

Table 6.10 — Production Deployment: Executive Summary

6.3.5.2 Generation Pipeline Performance

The 33 runs split evenly between completed and orphaned states, with a smaller failed cohort. Orphaned runs are an artifact of stopping containers during development rather than a runtime defect, but they highlight the absence of a startup reconciliation sweep to mark stale in-progress jobs as failed.

Status	Count	Share
completed	13	39.4 %
processing (orphaned / interrupted)	13	39.4 %
failed	7	21.2 %

Table 6.11 — Pipeline Outcome Distribution (n = 33)

Decomposing successful runs by stage confirms the estimate from Section 6.3.4 that asset generation dominates: it accounts for roughly 79 % of total wall-clock time on an average successful run (218 s of 277 s), with planning contributing about 15 % and the Godot export about 3 %. The code stage records only ~6ms because code artifacts are produced during the assets phase; the recorded step is a pipeline checkpoint rather than a separate computation. The build/export stage is highly deterministic ($\sigma = 0.72$ s), as expected for a fixed Godot headless export, while planning latency varies roughly $15\times$ (10.5 s $\rightarrow$ 158.6 s), reflecting LLM inference variance under provider load.

Stage	n	Mean	Median	p90	p95	Min	Max	Std dev
Planning (LLM game plan)	29	42.4	33.0	82.5	85.3	10.5	158.6	31.7
Assets (sprite / audio)	13	218.4	211.3	324.2	416.3	56.7	416.3	83.4
Code (handoff checkpoint)	13	0.006	0.006	0.007	0.007	0.006	0.007	~0
Building (Godot export + upload)	13	9.4	9.6	10.1	10.4	7.5	10.4	0.7

Table 6.12 — Per-Stage Latency, Completed Executions (seconds)

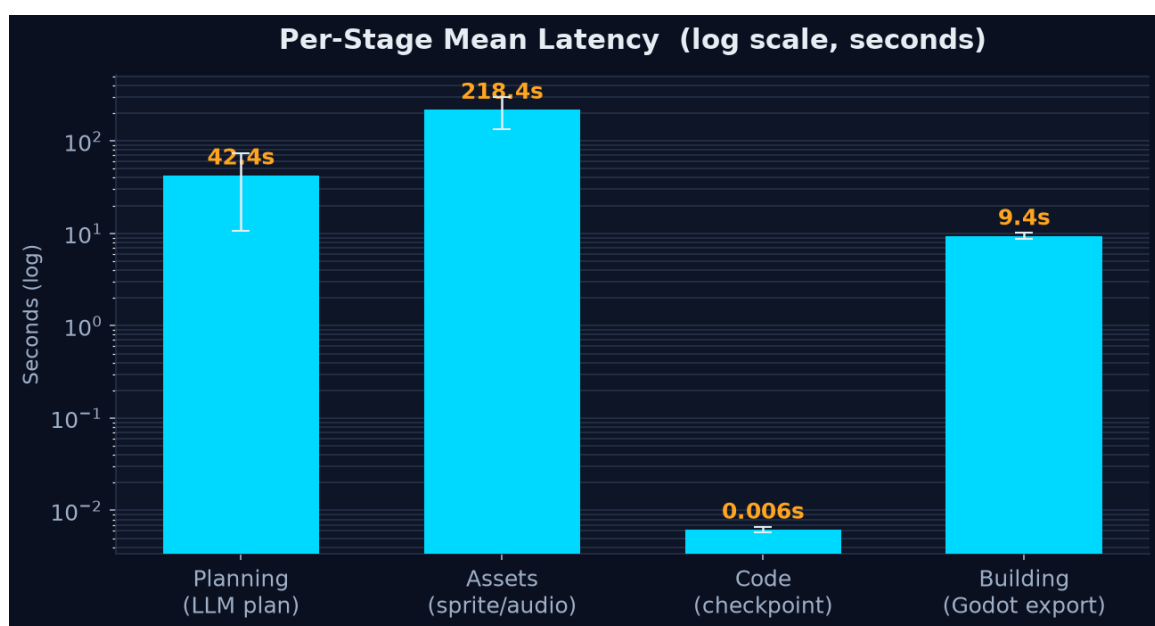


Figure 6.7 — Production: Per-Stage Mean Latency (log scale)

End-to-end latency for the 13 completed runs averages 276.5 s (median 268.2 s, p90 $\approx$ 402.4 s). Failed runs terminate after a comparable mean time (274.2 s) but with far greater variance, because a failure can occur early (32.8 s) or after a long asset timeout (621.5 s).

Outcome	n	Mean	Min	Max	Std dev
completed	13	276.5	109.7	453.1	92.9
failed	7	274.2	32.8	621.5	175.2

Table 6.13 — End-to-End Latency by Outcome (seconds)

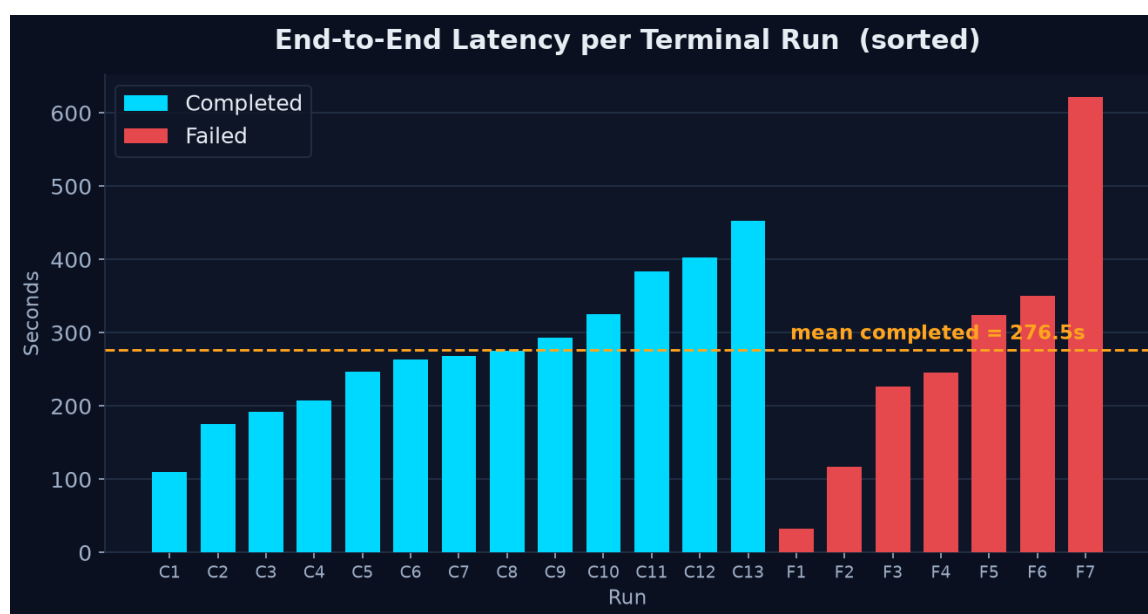


Figure 6.8 — Production: End-to-End Latency per Terminal Run

Every one of the seven hard failures occurred in the asset-generation stage — the longest-running, external-API-dependent step. Five of seven are availability or timeout issues on the asset service; the remaining two are LLM output-format robustness issues (non-strict JSON emitted by the model). The planning, code, and building stages recorded zero failures. Retry-with-backoff on asset calls plus schema-constrained or repair-parsing of LLM responses would address 100 % of the observed failures.

Root cause (from job error logs)	Occurrences
Asset-service request aborted due to timeout	3
fetch failed (asset service unreachable / connection reset)	2

Asset service 500 — malformed JSON in LLM response	1
Asset service 500 — failed to parse <MANIFEST> from LLM response	1

Table 6.14 — Pipeline Failure Causes (all 7 in the assets stage)

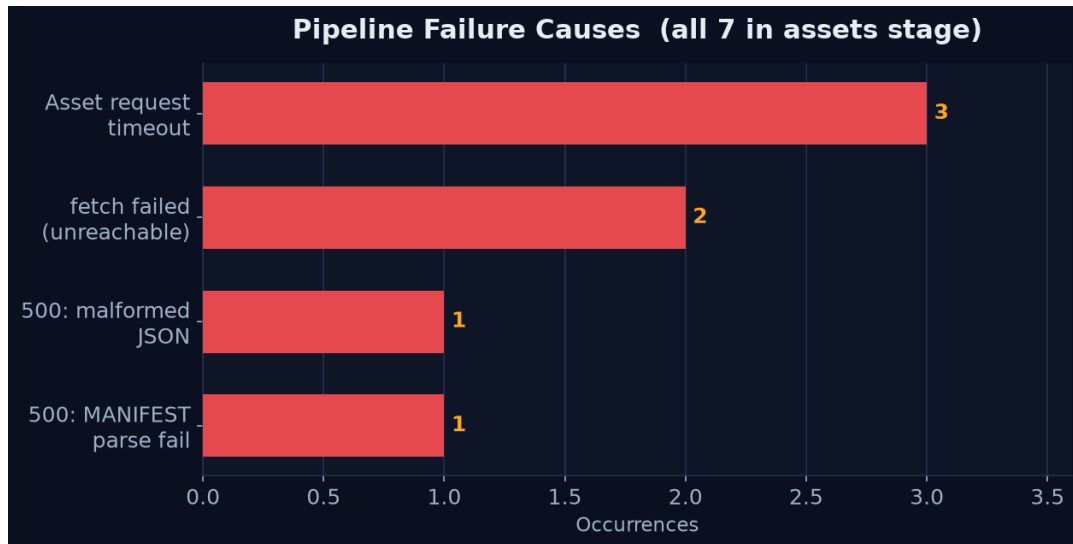


Figure 6.9 — Production: Pipeline Failure Causes

6.3.5.3 Build & Deliverable Metrics

The 13 completed builds produced 78 output artifacts, all targeting web (Godot 4.x HTML5 export). The average playable bundle is 67.7 MB (min 35.8 MB, max 80.5 MB) across six files. The Godot web runtime (index.wasm + index.js) is byte-identical on every build, so the game data pack (index.pck) is the only truly variable artifact.

Artifact	Count	Avg size	Total	Notes
index.wasm	13	35.38 MB	459.9 MB	Godot 4 web runtime (identical each build)
index.pck	13	31.91 MB	414.9 MB	Game data — the only variable artifact
index.js	13	331 KB	4.3 MB	Godot JS bootstrap
index.png + index.icon.png	26	13.6 KB	0.35 MB	Splash + icon
index.html	13	4.85 KB	0.06 MB	Loader page

Table 6.15 — Deliverable Bundle Composition (13 builds, 78 artifacts)

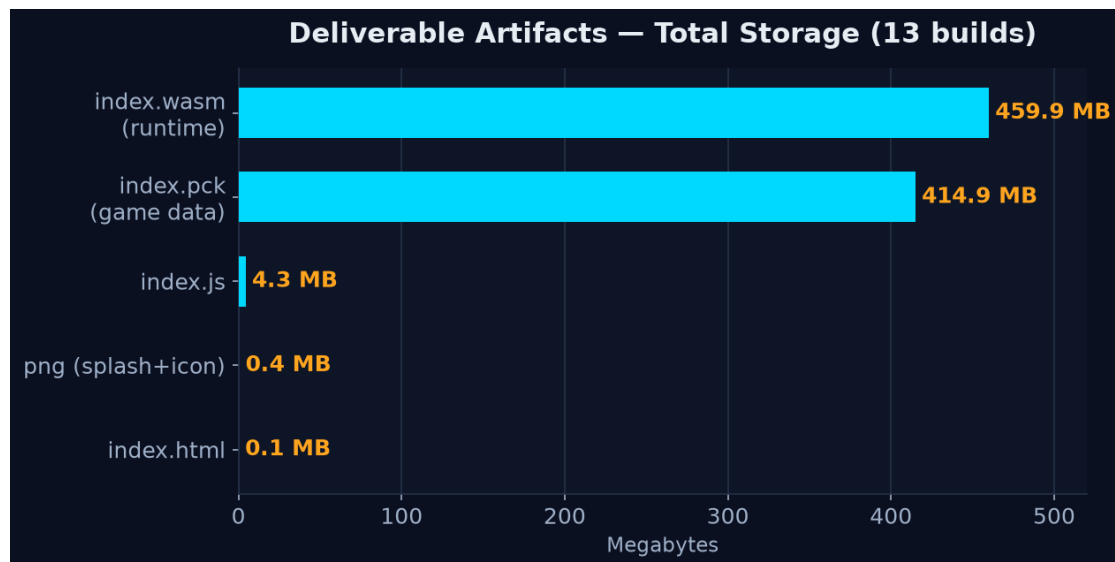


Figure 6.10 — Production: Deliverable Artifacts by Total Storage

6.3.5.4 Content Generation Metrics

The platform generated 154 asset records, all in the generated state. Enemy sprites, obstacle tilesets, and audio clips are the most numerous, reflecting the multi-enemy, multi-level plans typical of the observation window. By MIME type the assets divide into 119 image/png, 18 audio/mpeg, and 17 audio/wav files. Player sprites additionally carry animation metadata (on average 6 animations of 16 frames at 12 fps: idle, run, jump, double jump, fall, and die).

Asset type / entity	Count
Audio (SFX + music)	35
Sprite — enemy	29
Tileset — obstacle	27
Sprite — player	14
Background — environment	14
Sprite — collectible	13
Sprite — goal	13
Sprite — hazard	9

Table 6.16 — AI-Generated Assets by Type / Entity (n = 154)

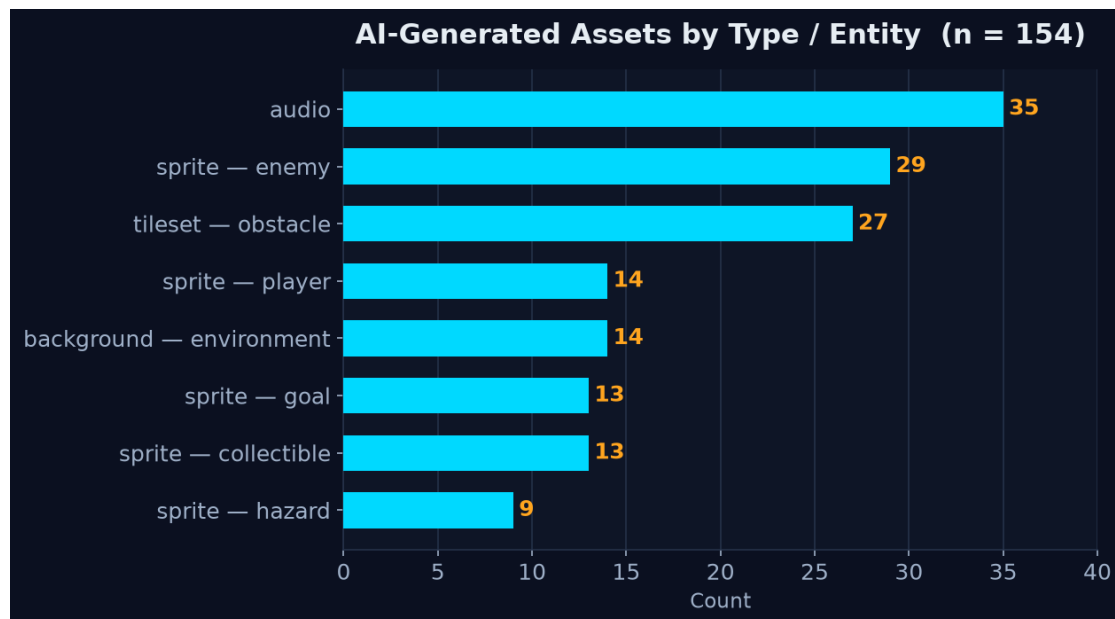


Figure 6.11 — Production: AI-Generated Assets by Type

The conversational planning interface recorded 36 chats totaling 280 messages (7.8 messages per chat on average, max 12), indicating that requirement gathering converges in under eight turns. The planner produced 29 structured game plans averaging 4.3 KB each (range 2.7–7.6 KB). Of the 36 projects, 13 completed, 13 remained running (stale), 7 failed, and 3 were drafts, with an average of 12 projects per user (max 24 on the primary test account).

6.3.5.7 Reconciliation with Estimated Benchmarks and Key Findings

The production measurements broadly validate the estimated benchmarks of Sections 6.3.1–6.3.4 while sharpening several numbers. Isolated planner latency (48.6 s) and production planner latency (42.4 s) agree closely. The estimated end-to-end times, however, were optimistic: real asset generation is slower and more variable than projected, pushing the measured median (268 s) above the estimated p50 (~190 s) and raising asset generation's share of wall time from the estimated 60–70 % to about 79 %. Conversely, the Godot export proved faster than estimated (9.4 s measured vs. an assumed ~25 s).

Metric	Ch. 6.3 benchmark / estimate	Production (measured)	Assessment
Planner latency (mean)	48.6 s (isolated benchmark)	42.4 s	Confirmed — close
Code assembly	0.38 s (isolated benchmark)	0.006 s (pipeline checkpoint)*	Context differs
Asset generation (mean)	≈ 124 s (implied by p50 estimate)	218.4 s	Under-estimated
Builder / Godot export	≈ 25 s (estimate)	9.4 s	Over-estimated
End-to-end p50 / median	≈ 190 s (estimate)	268.2 s	Under-estimated
End-to-end p90	≈ 230 s (estimate)	402.4 s	Under-estimated
Asset share of wall time	60–70 % (estimate)	≈ 79 %	Higher in practice
End-to-end total range	2.5–5 min (Ch. 1 / 7)	1.8–7.6 min	Consistent

Table 6.21 — Estimated (Ch. 6.3) vs. Measured (Production) Performance

6.4 Limitations and Known Issues

Game Forge is a working end-to-end system, but several deliberate scoping decisions and known constraints shape what it can and cannot produce. This section documents them honestly, grouped by subsystem. The recurring design theme is a preference for guaranteed playable output over open-ended flexibility: wherever the system had to choose, it constrains generation to a verified set of parts rather than risk producing a broken game.

6.4.1 Game Scope and Design Generation

6.4.1.1 Only Three Game Archetypes

The planner supports exactly three archetypes — platformer, top-down, and endless-runner. Any prompt describing a genre outside these three (RPG, puzzle, tower-defense, racing, fighting, and so on) is force-fit into the nearest supported archetype rather than rejected. This is a deliberate scoping decision that keeps every generated game within a verified template family. The pipeline is structured to extend later — the asset service already carries perspective presets for additional archetypes — but as it stands the genre coverage is narrow.

6.4.1.2 Fixed Behavior and Weapon Catalog

Enemy behaviors, player weapons, and win conditions are all drawn from a fixed catalog defined by archetype. A behavior the plan references but that is not in the catalog is silently dropped; weapons are limited to melee and ranged; and an unsupported win condition is coerced to a safe per-archetype default. This means novel or emergent mechanics a user describes cannot be realized — the game is limited to combinations of pre-written, Godot-verified parts. Optional LLM-authored custom-behavior and script-override channels exist for advanced cases, but they are validated and dropped if malformed, so they can never break a build.

6.4.1.3 Generic Fallback Plan

If the language model fails to return a usable plan after two attempts, the planner falls back to a generic hard-coded default plan, retaining the user's prompt only as a short description string. In that situation the user can receive a generic game bearing little relation to their request. The fallback is a last resort that guarantees the pipeline always returns something buildable rather than erroring out entirely.

6.4.2 Visual and Audio Asset Generation

6.4.2.1 Asset Generation Is the Pipeline Bottleneck

Image and audio calls to Stability AI are issued sequentially — one image per entity plus terrain tiles, a background, and a music track — so total asset time scales linearly with the number of entities and dominates the end-to-end latency of a generation. Code assembly, by contrast, runs in parallel and is effectively free. Batching and caching of asset calls are the identified optimizations for reducing overall generation time.

6.4.2.2 Dependence on External Image Credits

Asset generation requires a valid Stability AI key with available quota. When the key is missing or the quota is exhausted, per-asset generation fails and the game falls back to bundled placeholder sprites — a silent visual downgrade. A configuration flag allows the stack to run in a static-asset mode entirely without image credits for testing and demonstration.

6.4.2.3 No Frame-by-Frame Sprite Animation

The asset pipeline produces a single still image per entity rather than a spritesheet. All in-game motion is animated procedurally in Godot at runtime, and transparent cutouts rely on the provider's background-removal step over a solid-color prompt. This is intentional — it avoids the cost and unreliability of true multi-frame generation — but it means generated games cannot display genuine frame-by-frame character animation, and an imperfect cutout can occasionally leave a halo or clip the subject.

6.4.2.4 Art Style Is Not Always Exact

The requested art style is not always honored precisely. Some style labels are remapped — for example "hand-drawn" is served through a full-color comic-book preset and monochrome line-art is avoided — so the delivered look can differ from the label the user chose. These substitutions are deliberate and are made to keep output full-color and visually usable.

6.4.2.5 No Asset Caching

There is no caching or reuse of generated assets — identical prompts regenerate from scratch on every run, wasting API credits and time on re-rolls and on similar games. Caching by visual description is identified as future work.

6.4.3 Code Assembly

6.4.3.1 Template-Bounded Generation

The code service performs no LLM code generation; it assembles pre-written, Godot-verified GDScript fragments selected by the plan. This is the system's core reliability guarantee — assembled projects almost always compile and run — but it also means the universe of possible games is bounded by the archetype libraries. Anything that cannot be expressed through the catalog's behaviors and weapons cannot be built.

6.4.3.2 The Repair Loop Degrades Rather Than Repairs

When validation fails, the retry loop strips the offending file's authored extras and re-assembles; after the retry cap it proceeds despite the failure. This keeps the build alive but does so by silently removing authored content, so the final game can be simpler

than the plan described. Built-in behaviors are never the cause of such a failure, so the core game is preserved.

6.4.3.3 Bounded Level Variety

Level layouts are generated procedurally from a limited set of sections, room, and pattern templates driven by a seeded random generator. Variety is real but bounded by the template library, so layouts can begin to feel repetitive across many generations. A stored seed makes each run reproducible while keeping successive generations different.

6.4.4 Build and Export

6.4.4.1 Environment-Sensitive Godot Export

Full HTML5 export is sensitive to the environment and pinned to a specific Godot CLI version, and it is retried only once before failing. A toolchain mismatch or a transient export error can therefore fail the entire build. A skip-export stub mode is provided so the rest of the orchestration can be exercised without a real Godot export.

6.4.5 Orchestration, Scale, and Reliability

6.4.5.1 No Partial Recovery or Cancellation

Any stage failure fails the whole generation and discards partial artifacts, so a late-stage error throws away an already-successful plan and its assets. Users also cannot cancel an in-flight build. Failure states are marked cleanly and surfaced to the interface.

6.4.5.2 Polling-Based Progress and Best-Effort Status

Build progress is delivered by client polling with exponential backoff rather than a live stream, so updates lag by the poll interval and there is no live log feed. In addition, the cross-record status updates between the job, build, and project documents are best-effort and swallow errors, so their status can briefly drift out of sync if an update call fails. Both are simple, robust choices whose terminal states converge in the common path.

6.4.6 Authentication and Security

6.4.6.1 *Shared Inter-Service Secret*

Services authenticate one another with a single shared static secret, and an empty secret disables that check entirely — the development default. There is no per-service identity or mutual TLS, so a leaked secret would be authenticated as any service. This is adequate inside a trusted internal network, and the production configuration refuses to start with the secret unset.

6.4.6.2 *Publicly Addressable Game Files*

Generated game files are served publicly by game ID with no ownership check (downloads are verified only when a user session is present), so anyone who knows or guesses a game's URL can play it. The game ID is a random UUID that is unguessable in practice, and public playback is desirable for sharing.

Chapter 7: Conclusion and Future Work

7.1 Summary of Achievements

This project sets out to answer a deceptively simple question: can a natural-language prompt be turned into a complete, playable video game with no human in the loop? Game Forge demonstrates that it can. The system takes a single English description from a user, guides them through a short design conversation, and produces a fully playable Godot 4 HTML5 game — sprites, background art, terrain, music, level layout, enemy behavior, and win conditions included — that runs directly in the browser and can be downloaded as a self-contained bundle.

The concrete achievements of the project are as follows:

A complete, end-to-end generation pipeline. We designed and implemented a ten-service platform that carries a prompt through five distinct stages — design → planning → assets + code (in parallel) → build. Each stage is an independently deployable service, and the entire flow is orchestrated by a single stateless coordinator. From the

user's perspective, the experience is one click: they confirm a design and receive a running game roughly two-and-a-half to five minutes later.

A robust microservice architecture. The platform is composed of seven Node.js (ES-module) services and one Python FastMCP server, wired together with Docker Compose. We established a set of design principles that recur throughout the codebase and that proved essential to keeping a system of this size coherent: a shared provider abstraction for LLMs (so Groq and OpenRouter can each be selected by environment variable), MinIO object storage as the single shared filesystem between stages, MongoDB as the single shared state store, and a shared-secret header for inter-service authentication. No service streams files directly to another; they communicate through storage and through the database, which keeps each service testable and independently restartable.

Separation of AI creativity from deterministic assembly. One of the most important architectural decisions was to not let a language model write game code. Instead, the planner LLM chooses behaviors and weapons by name from a fixed, engine-verified catalogue and fills in numeric parameters, while the code service performs pure, deterministic assembly of GDScript from a pre-written template library — in roughly 0.4 seconds and with no LLM call at all. This split gives us the creativity of generative AI where it adds value (game design, art prompts) and the reliability of conventional software everywhere correctness matters (the actual runtime code). Anything the plan references that are not in the library is simply dropped before assembly, so the model can never inject invalid or unsafe code.

A solution to the naming-and-ordering problem. A central technical challenge was that the code service finishes in under half a second while asset generation takes minutes — yet the generated code must reference asset files that do not yet exist. We solved this with a name-as-contract design: the entity's plan name (e.g. hero, skeleton, coin) is the shared key that every service independently derives the same way. The code hard-codes `res://assets/hero.png` before any asset exists; the asset service stores each generated file under a random storage key but records it in a MongoDB Asset document keyed on the human name; and the builder — the join point — resolves that document back to the exact filename the code expects. The build never

starts until both the fast and slow branches complete, because they are awaited together, so by the time the game is exported every referenced file has been laid down.

Graceful degradation throughout. Every stage of the pipeline is designed to survive partial failure. A failed asset never breaks the build — the game falls back to flat-colored rectangles. A malformed LLM plan falls back to a hard-coded default plan. A Godot parse error strips only the offending authored snippet and re-assembles, up to a retry cap. A misclassified win condition is coerced to one the engine implements, guarding against the "silently unwinnable game" failure mode. This defensive posture means the system almost always produces something playable rather than failing outright.

Procedural, physics-aware level generation. Rather than emitting static layouts, we built seeded procedural level generators for each game archetype (platformer, top-down, endless-runner). The platformer generator, for example, derives a jump-and-gap reachability envelope from the game's actual physics tuning and stitches levels from a library of hand-authored, guaranteed-reachable prefab chunks arranged into a difficulty ramp — so a generated level is never impossible to complete.

A verified, tested system. The platform is backed by 621 automated tests across all services, using Vitest and Supertest for the Node services and pytest for the Python MCP server, with every external dependency (LLMs, Stability AI, Godot, MongoDB, MinIO) mocked so that no test requires real secrets, API quota, or network access. We complemented this with performance benchmarks that characterized each stage: the code service assembles a project in 0.36–0.53 seconds regardless of size, and the full pipeline's latency is shown to be almost entirely asset-bound.

Taken together, these achievements deliver working, reproducible, and thoroughly tested proof that generative AI can be combined with deterministic software engineering to automate the creation of complete, playable games from natural language

7.2 Lessons Learned

Building a system of this complexity taught us as much about software architecture and the practical realities of working with generative AI as it did about game generation itself.

Constrain the language model; don't trust it to be correct. The single most valuable lesson was that reliable systems built on LLMs come from *narrowing what the model is allowed to do*, not from prompting it more cleverly. Our early instinct might have been to have the model generate code directly, but this is fragile and unverifiable. By reducing the model's role to picking from a fixed catalogue of names and filling numeric parameters — and by validating and dropping anything outside that catalogue — we turned an unpredictable component into a well-behaved one. Similarly, keeping the asset-prompt LLM *out* of the tool-calling loop (code, not the model, decides what to generate and how many times) eliminated an entire class of double-invocation and formatting bugs.

Wall-clock ordering is not dependency ordering. Because the code branch finishes ~200× faster than the asset branch, reasoning about the pipeline as "A then B then C" invites broken assumptions. Making the dependency explicit — awaiting both branches together and designating the builder as the join point — proved far more robust than coordinating services by timing.

Design for failure from the start, not as an afterthought. Because we are chaining together non-deterministic components (LLMs) and external APIs with quotas and latency variance (Stability AI, Godot), failure is the normal case, not the exception. The habit of asking "what happens if this returns null / times out / produces garbage?" at *every* boundary — and always having a fallback — is what makes the difference between a demo that works once and a system that works repeatedly. The fallbacks (placeholder sprites, default plans, snippet-stripping retries) were not polish; they were load-bearing.

Decouple services through shared storage and state, not direct calls. Choosing MinIO as a shared filesystem and MongoDB as shared state — rather than having services stream data to one another — paid off repeatedly. It meant each service could be developed, tested, and restarted in isolation, and it made the flow of data easy to inspect (you can look in the bucket and see exactly what each stage produced). The one stateful coordinator that holds *no state of its own* and mutates everything through the backend's internal routes proved to be a clean pattern.

A shared naming convention beats runtime negotiation. Five services agree on where a file lives without ever talking to each other, simply by applying the same `safeName()` rule to the same plan name. Conventions, enforced consistently, remove entire categories of coordination logic.

Practical friction with real tooling. Working with the Godot CLI, the FastMCP protocol, and undici's HTTP client surfaced concrete lessons that generic tutorials do not teach: that FastMCP requires arguments nested under a `params` object, that an `AbortSignal` alone cannot override undici's aggressive 30-second header timeout (forcing us to configure a custom agent for the multi-minute asset and build stages), and that Godot's threaded WASM export requires specific cross-origin isolation headers (COOP/COEP/CORP) to run in an `iframe`. These small, hard-won details consumed real time and reinforced the value of writing them down.

Test coverage percentage is a measurement, not a goal. Our coverage numbers vary by service, from about 32% on the browser frontend to about 96% on the orchestration server, and we learned to treat these as honest measurements rather than targets to be gamed. We concentrated testing effort on the surfaces where correctness matters most — validators, the orchestration pipeline, the naming resolver, middleware, and the code assembler — which are all well covered, while the browser UI remains the least-exercised area in this first pass. Being explicit about *what* is and isn't covered, and *why*, is more useful than chasing a single headline number.

Honest evaluation reveals real limitations. Our benchmarking discovered a genuine weakness — the planner misclassifies certain endless-runner prompts as

platformers — that we chose to document rather than hide. Building the measurement infrastructure to *find* such issues was as important as the features themselves.

7.3 Future Work

Game Forge is a solid foundation, but there are many directions in which the platform can be extended and strengthened. We group the most promising avenues below.

Broaden the range of generatable games. The system supports only three archetypes today. The architecture makes new ones additive — each is a self-contained directory of a manifest, runtime scripts, behaviors, weapons, and a level generator — so additions such as puzzle games, shoot-'em-ups, tower-defense, and simple RPGs would each widen the space of describable games.

Improve plan classification and design fidelity. The endless-runner-as-platformer misclassification noted earlier should be addressed — for example, with better few-shot examples, a fine-tuned classifier, or a confirmation step that surfaces the detected archetype to the user for correction. More broadly, richer plan schemas could capture narrative, progression, and difficulty preferences that the current design conversation does not yet express.

Animated and stylistically consistent assets. Building on the single-still-image limitation noted above, a significant enhancement would be true generated sprite sheets or animation frames, plus mechanisms to enforce a consistent art style across a game's assets (shared palette, coherent character proportions).

End-to-end and load testing. The test suite is strong at the unit and service-integration level but, by design, mocks the live LLMs, Stability AI, Godot, and databases. Priorities here include a Playwright browser end-to-end test that drives the real UI through a full generation, load and concurrency testing to understand how the system behaves under many simultaneous builds, and a CI stage that periodically exercises the *live* external services to catch integration drift.

Performance and cost optimization. Because the pipeline is almost entirely asset-bound, parallelizing asset generation across entities, caching or reusing assets

between similar games, streaming partial progress (e.g. letting the user play with placeholder art while final art generates), and offering a "fast draft" mode would all meaningfully improve the user experience and reduce costs.

Iterative editing and refinement. Today the flow is one-shot: describe, confirm, generate. A powerful extension would be *conversational iteration* — allowing a user to say, "make the enemies faster," "add a boss," or "change the theme to space" and have the system re-plan and re-build only what changed. The seeded, deterministic level generation already supports exact reproduction, which is a natural foundation for targeted re-rolls.

Scalability and production hardening. For real-world deployment beyond a single Docker Compose host, the platform would benefit from a proper job queue (rather than fire-and-forget HTTP), horizontal scaling of the asset and builder workers, orchestration under Kubernetes, observability (structured metrics, tracing across the service hops), and rate-limiting and quota management per user.

Richer user-facing features. Finally, there is a long tail of product features that would increase the platform's value: user galleries and sharing of generated games, remixing another user's game as a starting point, exporting to native desktop or mobile targets in addition to HTML5, and in-browser fine-tuning controls (difficulty sliders, level count, palette pickers) layered on top of the natural-language flow.

Collectively, these directions would take Game Forge from a compelling proof of concept to a robust, scalable, and genuinely delightful tool for turning ideas into playable games.

References

1. [1] Rosebud AI, "Rosebud AI: Free AI Game Maker — Create Games Online," Rosebud AI, Inc. [Online]. Available: <https://rosebud.ai/>.
2. [2] H. Al-Ali, O. Jarrah, and S. Kiswani, "Launch YC: Playabl.ai — TikTok for User-Generated Games," Y Combinator. [Online]. Available: <https://www.ycombinator.com/launches/QeI-playabl-ai-tiktok-for-user-generated-games>.
3. [3] "Playabl Review: Free AI Game Builder for Everyone 2026," HokAI. [Online]. Available: <https://hokai.io/hub/tools/playabl>.
4. [4] Astrocade, "About Astrocade — Our Mission to Democratize Game Creation," Astrocade, Inc. [Online]. Available: <https://www.astrocade.com/about-us>.
5. [5] Astrocade, "Astrocade 2.0: World's First Agentic AI Game Creation Platform," Astrocade Blog, Jul. 30, 2025. [Online]. Available: <https://www.astrocade.com/blog/astrocade-2-worlds-first-agentic-ai-game-creation-platform>.
6. [6] J. Batchelor, "Astrocade Raises \$56m to Expand AI-Powered Games Creation Platform," PocketGamer.biz, May 7, 2026. [Online]. Available: <https://www.pocketgamer.biz/astrocade-raises-56m-to-expand-ai-powered-games-creation-platform/>.
7. [7] StoneLeap, "StoneLeap AI — AI First Game Development Platform," StoneLeap AB. [Online]. Available: <https://www.stoneleap.ai/>.
8. [8] Ludo.ai, "Ludo.ai — AI Sprite Generator and Game Asset Pipeline," Ludo.ai. [Online]. Available: <https://ludo.ai/>.
9. [9] Ludo.ai, "Turn Game Ideas into Playable Prototypes with Ludo.ai's Playable Generator," Ludo.ai. [Online]. Available: <https://ludo.ai/features/playable-generator>.
10. [10] Anthropic, "Introducing the Model Context Protocol," Anthropic. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>.

- 11.[11] GDevelop, "GDevelop: Free, Fast, Easy Game Engine — No-Code, AI-Assisted," GDevelop. [Online]. Available: <https://gdevelop.io/>.
- 12.[12] GDevelop, "GDevelop's AI Agent: Create Game Features Automatically," GDevelop Blog, Oct. 9, 2025. [Online]. Available: <https://gdevelop.io/blog/make-games-with-ai-agent-gdevelop-automated-prompt>.
- 13.[13] GDevelop, "Ask AI (Build/Chat with AI)," GDevelop Documentation. [Online]. Available: <https://wiki.gdevelop.io/gdevelop5/interface/ai/>.
- 14.[14] GitHub, "GitHub Copilot — Your AI Pair Programmer," GitHub, Inc. [Online]. Available: <https://github.com/features/copilot>.
- 15.[15] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Cham, Switzerland: Springer, 2016. [Online]. Available: <http://pcgbook.com>.
- 16.[16] Meta Open Source, "React – The Library for Web and Native User Interfaces." [Online]. Available: <https://react.dev/>.
- 17.[17] "Vite – Next Generation Frontend Tooling." [Online]. Available: <https://vite.dev/>.
- 18.[18] Tailwind Labs, "Tailwind CSS – Rapidly Build Modern Websites Without Ever Leaving Your HTML." [Online]. Available: <https://tailwindcss.com/>.
- 19.[19] Motion, "Motion (formerly Framer Motion): A Modern Animation Library for React and JavaScript." [Online]. Available: <https://motion.dev/>.
- 20.[20] OpenJS Foundation, "Node.js – Run JavaScript Everywhere." [Online]. Available: <https://nodejs.org/>.
- 21.[21] OpenJS Foundation, "Express – Fast, Unopinionated, Minimalist Web Framework for Node.js." [Online]. Available: <https://expressjs.com/>.
- 22.[22] "Mongoose ODM v8 – Elegant MongoDB Object Modeling for Node.js." [Online]. Available: <https://mongoosejs.com/>.
- 23.[23] MongoDB, Inc., "MongoDB: The Developer Data Platform." [Online]. Available: <https://www.mongodb.com/>.
- 24.[24] MinIO, Inc., "MinIO: S3-Compatible High-Performance Object Storage." [Online]. Available: <https://min.io/>.

25. [25] Groq, Inc., "GroqCloud – Fast AI Inference." [Online]. Available: <https://groq.com/>.
26. [26] OpenRouter, "OpenRouter – A Unified Interface for LLMs." [Online]. Available: <https://openrouter.ai/>.
27. [27] Zhipu AI (Z.ai), "GLM – Z.ai Large Language Models." [Online]. Available: <https://z.ai/>.
28. [28] Meta AI, "Llama 3.3." [Online]. Available: <https://www.llama.com/>.
29. [29] Qwen Team, Alibaba Cloud, "Qwen3." [Online]. Available: <https://qwenlm.github.io/blog/qwen3/>.
30. [30] OpenAI, "Introducing gpt-oss." [Online]. Available: <https://openai.com/index/introducing-gpt-oss/>.
31. [31] DeepSeek, "DeepSeek – Into the Unknown." [Online]. Available: <https://www.deepseek.com/>.
32. [32] Stability AI, "Stable Image Core – Stability AI Image Models." [Online]. Available: <https://stability.ai/stable-image>.
33. [33] Stability AI, "Introducing Stable Audio 2.0." [Online]. Available: <https://stability.ai/news/stable-audio-2-0>.
34. [34] Prefect, "FastMCP – The Fast, Pythonic Way to Build MCP Servers." [Online]. Available: <https://gofastmcp.com/>.
35. [35] Godot Engine, "Godot Engine – Free and Open Source 2D and 3D Game Engine." [Online]. Available: <https://godotengine.org/>.
36. [36] Docker, Inc., "Docker: Accelerated Container Application Development." [Online]. Available: <https://www.docker.com/>.
37. [37] Docker, Inc., "Docker Compose." [Online]. Available: <https://docs.docker.com/compose/>.
38. [38] GitHub, Inc., "Working with the Container Registry." [Online]. Available: <https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-container-registry>.
39. [39] GitHub, Inc., "GitHub Actions – Automate Your Workflow from Idea to Production." [Online]. Available: <https://github.com/features/actions>.
40. [40] "ESLint – Find and Fix Problems in Your JavaScript Code." [Online]. Available: <https://eslint.org/>.
41. [41] "JSON Web Tokens – jwt.io." [Online]. Available: <https://jwt.io/>.

42. [42] "bcryptjs – Optimized bcrypt in JavaScript." [Online]. Available: <https://www.npmjs.com/package/bcryptjs>.
43. [43] "Helmet.js – Secure Express Apps by Setting HTTP Response Headers." [Online]. Available: <https://helmetjs.github.io/>.
44. [44] "express-rate-limit – Basic Rate-Limiting Middleware for Express." [Online]. Available: <https://www.npmjs.com/package/express-rate-limit>.
45. [45] "xss-clean – Middleware to Sanitize User Input Coming from Request Body, Query, and Params." [Online]. Available: <https://www.npmjs.com/package/xss-clean>.
46. [46] "express-mongo-sanitize – Sanitize Your Express Payload to Prevent MongoDB Operator Injection." [Online]. Available: <https://www.npmjs.com/package/express-mongo-sanitize>.
47. [47] "Vitest – Next Generation Testing Framework." [Online]. Available: <https://vitest.dev/>.
48. [48] "pytest – Helps You Write Better Programs." [Online]. Available: <https://docs.pytest.org/>.
49. [49] Testing Library, "React Testing Library." [Online]. Available: <https://testing-library.com/docs/react-testing-library/intro/>.
50. [50] "jsdom – A JavaScript Implementation of WHATWG DOM and HTML Standards." [Online]. Available: <https://github.com/jsdom/jsdom>.
51. [51] "SuperTest – Super-Agent Driven Library for Testing Node.js HTTP Servers." [Online]. Available: <https://www.npmjs.com/package/supertest>.
52. [52] "pytest-asyncio – Pytest Support for asyncio." [Online]. Available: <https://pytest-asyncio.readthedocs.io/>.
53. [53] "pytest-cov – Coverage Plugin for pytest." [Online]. Available: <https://pytest-cov.readthedocs.io/>.
- 54.

Appendix A:

Google drive containing all diagrams &
GitHub organization Link. (QR Code).

